

5. Vue2 核心模块源码解析

课程资料



5.vue2 源码.zip

2.04MB



课程目标



- 初级：
 - 掌握 Vue2 数据驱动的思路；
 - 掌握双端比较的使用；
- 中级：
 - 掌握 Vue2 的整体源码的核心执行流程；
 - 能够掌握 Vue2 diff 的使用；
- 高级：
 - 深入理解 Vue2 核心源码，熟练掌握 Vue2 完整的执行逻辑；
 - 完整掌握 Vue2 diff 的执行思路；

课程大纲

1. 前置知识；
2. 数据驱动；
3. Vue2 Diff 算法；

课程内容

前置知识

Flow（只是作为阅读 Vue2 源码前置知识的一个简单了解）

Flow 是 facebook 出品的 JavaScript 静态类型检查工具。Vue.js 的源码利用了 Flow 做了静态类型检查，也就是文件顶部出现的

```
1  /* @flow */
```

使用Flow的原因

1. JavaScript 是动态类型语言，它的灵活性有目共睹，但是过于灵活的副作用是很容易就写出非常隐蔽的隐患代码，在编译期甚至看上去都不会报错，但在运行阶段就可能出现各种奇怪的 bug；
2. 类型检查是当前动态类型语言的发展趋势，可以帮助我们在编译期尽早发现（由类型错误引起的）bug，又不影响代码运行（不需要运行时动态检查类型），使编写 JavaScript 具有和编写 Java 等强类型语言相近的体验；
3. 项目越复杂就越需要通过工具的手段来保证项目的维护性和增强代码的可读性。Vue.js 在做 2.0 重构的时候，在 ES2015 的基础上，除了 ESLint 保证代码风格之外，也引入了 Flow 做静态类型检查。之所以选择 Flow，主要是因为 Babel 和 ESLint 都有对应的 Flow 插件以支持语法，可以完全沿用现有的构建配置，非常小成本的改动就可以拥有静态类型检查的能力；

Vue2.0选用Flow的具体原因，当然Vue3.0还是还是用TS重构了。

Flow 的工作方式

通常类型检查分成 2 种方式：

- 类型推断：通过变量的使用上下文来推断出变量类型，然后根据这些推断来检查类型；
- 类型注释：事先注释好我们期待的类型，Flow 会基于这些注释来判断；

类型推断

它不需要任何代码修改即可进行类型检查，最小化开发者的工作量。它不会强制你改变开发习惯，因为它会自动推断出变量的类型。这就是所谓的类型推断，Flow 最重要的特性之一。

通过一个简单例子说明一下：

```
1  /*@flow*/
2
3  function split(str) {
4    return str.split(' ')
5  }
```

```
6
7  split(11)
```

Flow 检查上述代码后会报错，因为函数 `split` 期待的参数是字符串，而我们输入了数字；

类型注释

如上所述，类型推断是 Flow 最有用的特性之一，不需要编写类型注释就能获取有用的反馈。但在某些特定的场景下，添加类型注释可以提供更好更明确的检查依据。

考虑如下代码：

```
1  /*@flow*/
2
3  function add(x, y){
4    return x + y
5  }
6
7  add('Hello', 11)
```

Flow 检查上述代码时检查不出任何错误，因为从语法层面考虑，`+` 既可以用在字符串上，也可以用在数字上，我们并没有明确指出 `add()` 的参数必须为数字。

在这种情况下，我们可以借助类型注释来指明期望的类型。类型注释是以冒号 `:` 开头，可以在函数参数，返回值，变量声明中使用。

如果我们在上段代码中添加类型注释，就会变成如下：

```
1  /*@flow*/
2
3  function add(x: number, y: number): number {
4    return x + y
5  }
6
7  add('Hello', 11)
```

现在 Flow 就能检查出错误，因为函数参数的期待类型为数字，而我们提供了字符串。

上面的例子是针对函数的类型注释。接下来我们来看看 Flow 能支持的一些常见的类型注释。

数组

```

1  /*@flow*/
2
3  var arr: Array<number> = [1, 2, 3]
4
5  arr.push('Hello')

```

数组类型注释的格式是 `Array<T>`，`T` 表示数组中每项的数据类型。在上述代码中，`arr` 是每项均为数字的数组。如果我们给这个数组添加了一个字符串，Flow 能检查出错误。

类和对象

```

1  /*@flow*/
2
3  class Bar {
4      x: string;           // x 是字符串
5      y: string | number;  // y 可以是字符串或者数字
6      z: boolean;
7
8      constructor(x: string, y: string | number) {
9          this.x = x
10         this.y = y
11         this.z = false
12     }
13 }
14
15 var bar: Bar = new Bar('hello', 4)
16
17 var obj: { a: string, b: number, c: Array<string>, d: Bar } = {
18     a: 'hello',
19     b: 11,
20     c: ['hello', 'world'],
21     d: new Bar('hello', 3)
22 }

```

类的类型注释格式如上，可以对类自身的属性做类型检查，也可以对构造函数的参数做类型检查。这里需要注意的是，属性 `y` 的类型中间用 `|` 做间隔，表示 `y` 的类型即可以是字符串也可以是数字。

对象的注释类型类似于类，需要指定对象属性的类型。

Null

若想任意类型 `T` 可以为 `null` 或者 `undefined`，只需类似如下写成 `?T` 的格式即可。

```
1  /*@flow*/
2
3  var foo: ?string = null
```

此时，foo 可以为字符串，也可以为 null。

目前我们只列举了 Flow 的一些常见的类型注释。如果了解所有类型注释，请移步 Flow 的[官方文档](#)。

Flow 在 Vue.js 源码中的应用

有时候我们想引用第三方库，或者自定义一些类型，但 Flow 并不认识，因此检查的时候会报错。为了解决这类问题，Flow 提出了一个 libdef 的概念，可以用来识别这些第三方库或者是自定义类型，而 Vue.js 也利用了这一特性。

在 Vue.js 的主目录下有 .flowconfig 文件，它是 Flow 的配置文件，文件内容为：

```
1  flow
2  |— compiler.js      # 编译相关
3  |— component.js     # 组件数据结构
4  |— global-api.js    # Global API 结构
5  |— modules.js       # 第三方库定义
6  |— options.js       # 选项相关
7  |— ssr.js           # 服务端渲染相关
8  |— vnode.js         # 虚拟 node 相关
```

可以看到，Vue.js 有很多自定义类型的定义，在阅读源码的时候，如果遇到某个类型并想了解它完整的数据结构的时候，可以回来翻阅这些数据结构的定义。

Vue 目录结构设计

```
1  src
2  |— compiler         # 编译相关
3  |— core             # 核心代码
4  |— platforms        # 不同平台的支持
5  |— server           # 服务端渲染
6  |— sfc              # .vue 文件解析
7  |— shared           # 共享代码
```

compiler

包含 Vue.js 所有编译相关的代码。它包括把模板解析成 ast 语法树，ast 语法树优化，代码生成等功能。

编译的工作可以在构建时做（借助 webpack、vue-loader 等辅助插件）；也可以在运行时做，使用包含构建功能的 Vue.js。显然，编译是一项耗性能的工作，所以更推荐前者——离线编译。

core

包含了 Vue.js 的核心代码，包括内置组件、全局 API 封装，Vue 实例化、观察者、虚拟 DOM、工具函数等等。

这里的代码可谓是 Vue.js 的灵魂，也是我们之后需要重点分析的地方。

platform

Vue.js 是一个跨平台的 类MVVM 框架，它可以跑在 web 上，也可以配合 weex 跑在 native 客户端上。platform 是 Vue.js 的入口，2 个目录代表 2 个主要入口，分别打包成运行在 web 上和 weex 上的 Vue.js。

server

Vue.js 2.0 支持了服务端渲染，所有服务端渲染相关的逻辑都在这个目录下。

这部分代码是跑在服务端的 Node.js，不要和跑在浏览器端的 Vue.js 混为一谈。

服务端渲染主要的工作是把组件渲染为服务器端的 HTML 字符串，将它们直接发送到浏览器，最后将静态标记"混合"为客户端上完全交互的应用程序。

sfc

通常我们开发 Vue.js 都会借助 webpack 构建，然后通过 .vue 单文件来编写组件。

这个目录下的代码逻辑会把 .vue 文件内容解析成一个 JavaScript 的对象。

shared

Vue.js 会定义一些工具方法，这里定义的工具方法都是会被浏览器端的 Vue.js 和服务端的 Vue.js 所共享的。

Vue源码构建

Vue.js 源码是基于 Rollup 构建的，它的构建相关配置都在 scripts 目录下。

构建脚本

Vue.js 源码构建的脚本如下：（只说跟build相关）

```
1  "build": "node scripts/build.js",
2  "build:ssr": "npm run build -- web-runtime-cjs,web-server-renderer",
3  "build:weex": "npm run build -- weex",
```

这里总共有 3 条命令，作用都是构建 Vue.js，后面 2 条是在第一条命令的基础上，添加一些环境参数。

当在命令行运行 `npm run build` 的时候，实际上就会执行 `node scripts/build.js`，接下来我们来看看它实际是怎么构建的。

构建过程

在 `scripts/build.js` 中：

```
1  let builds = require('./config').getAllBuilds()
2
3  // filter builds via command line arg
4  if (process.argv[2]) {
5    const filters = process.argv[2].split(',')
6    builds = builds.filter(b => {
7      return filters.some(f => b.output.file.indexOf(f) > -1 ||
8        b._name.indexOf(f) > -1)
9    })
10   } else {
11     // filter out weex builds by default
12     builds = builds.filter(b => {
13       return b.output.file.indexOf('weex') === -1
14     })
15   }
16   build(builds)
```

这段代码逻辑非常简单，先从配置文件读取配置，再通过命令行参数对构建配置做过滤，这样就可以构建出不同用途的 Vue.js 了。接下来我们看一下配置文件，在 `scripts/config.js` 中：

```
1  const builds = {
```

```
2 // Runtime only (CommonJS). Used by bundlers e.g. Webpack & Browserify
3 'web-runtime-cjs': {
4   entry: resolve('web/entry-runtime.js'),
5   dest: resolve('dist/vue.runtime.common.js'),
6   format: 'cjs',
7   banner
8 },
9 // Runtime+compiler CommonJS build (CommonJS)
10 'web-full-cjs': {
11   entry: resolve('web/entry-runtime-with-compiler.js'),
12   dest: resolve('dist/vue.common.js'),
13   format: 'cjs',
14   alias: { he: './entity-decoder' },
15   banner
16 },
17 // Runtime only (ES Modules). Used by bundlers that support ES Modules,
18 // e.g. Rollup & Webpack 2
19 'web-runtime-esm': {
20   entry: resolve('web/entry-runtime.js'),
21   dest: resolve('dist/vue.runtime.esm.js'),
22   format: 'es',
23   banner
24 },
25 // Runtime+compiler CommonJS build (ES Modules)
26 'web-full-esm': {
27   entry: resolve('web/entry-runtime-with-compiler.js'),
28   dest: resolve('dist/vue.esm.js'),
29   format: 'es',
30   alias: { he: './entity-decoder' },
31   banner
32 },
33 // runtime-only build (Browser)
34 'web-runtime-dev': {
35   entry: resolve('web/entry-runtime.js'),
36   dest: resolve('dist/vue.runtime.js'),
37   format: 'umd',
38   env: 'development',
39   banner
40 },
41 // runtime-only production build (Browser)
42 'web-runtime-prod': {
43   entry: resolve('web/entry-runtime.js'),
44   dest: resolve('dist/vue.runtime.min.js'),
45   format: 'umd',
46   env: 'production',
47   banner
48 },
```

```

49 // Runtime+compiler development build (Browser)
50 'web-full-dev': {
51   entry: resolve('web/entry-runtime-with-compiler.js'),
52   dest: resolve('dist/vue.js'),
53   format: 'umd',
54   env: 'development',
55   alias: { he: './entity-decoder' },
56   banner
57 },
58 // Runtime+compiler production build (Browser)
59 'web-full-prod': {
60   entry: resolve('web/entry-runtime-with-compiler.js'),
61   dest: resolve('dist/vue.min.js'),
62   format: 'umd',
63   env: 'production',
64   alias: { he: './entity-decoder' },
65   banner
66 },
67 // ...
68 }

```

这里列举了一些 Vue.js 构建的配置，对于单个配置，它是遵循 Rollup 的构建规则的。其中 entry 属性表示构建的入口 JS 文件地址，dest 属性表示构建后的 JS 文件地址。format 属性表示构建的格式，cjs 表示构建出来的文件遵循 [CommonJS](#) 规范，es 表示构建出来的文件遵循 [ES Module](#) 规范。umd 表示构建出来的文件遵循 [UMD](#) 规范。

以 `web-runtime-cjs-dev` 配置为例，它的 entry 是 `resolve('web/entry-runtime.js')`，先来看一下 resolve 函数的定义。

源码目录：scripts/config.js

```

1  const aliases = require('./alias')
2  const resolve = p => {
3    const base = p.split('/')[0]
4    if (aliases[base]) {
5      return path.resolve(aliases[base], p.slice(base.length + 1))
6    } else {
7      return path.resolve(__dirname, '../', p)
8    }
9  }

```

这里的 resolve 函数实现非常简单，它先把 resolve 函数传入的参数 p 通过 / 做了分割成数组，然后取数组第一个元素设置为 base。在我们这个例子中，参数 p 是 `web/entry-runtime.js`，那么 base 则为 web。base 并不是实际的路径，它的真实路径借助了别名的配置，我们来看一下别名配置的代码，在 `scripts/alias` 中：

```
1  const path = require('path')
2
3  const resolve = p => path.resolve(__dirname, '../', p)
4
5  module.exports = {
6    vue: resolve('src/platforms/web/entry-runtime-with-compiler'),
7    compiler: resolve('src/compiler'),
8    core: resolve('src/core'),
9    shared: resolve('src/shared'),
10   web: resolve('src/platforms/web'),
11   weex: resolve('src/platforms/weex'),
12   server: resolve('src/server'),
13   sfc: resolve('src/sfc')
14 }
15
```

很显然，这里 web 对应的真实的路径是 `path.resolve(__dirname, '../src/platforms/web')`，这个路径就找到了 Vue.js 源码的 web 目录。然后 resolve 函数通过 `path.resolve(aliases[base], p.slice(base.length + 1))` 找到了最终路径，它就是 Vue.js 源码 web 目录下的 `entry-runtime.js`。因此，`web-runtime-cjs-dev` 配置对应的入口文件就找到了。

它经过 Rollup 的构建打包后，最终会在 dist 目录下生成

`dist/vue.runtime.common.dev.js`。

Runtime Only VS Runtime + Compiler

通常我们利用 `vue-cli` 去初始化我们的 Vue.js 项目的时候会询问我们用 `Runtime Only` 版本的还是 `Runtime + Compiler` 版本。下面我们来对比这两个版本。

- Runtime Only

我们在使用 Runtime Only 版本的 Vue.js 的时候，通常需要借助如 webpack 的 `vue-loader` 工具把 `.vue` 文件编译成 JavaScript，因为是在编译阶段做的，所以它只包含运行时的 Vue.js 代码，因此代码体积也会更轻量；

- Runtime + Compiler

我们如果没有对代码做预编译，但又使用了 Vue 的 `template` 属性并传入一个字符串，则需要在客户端编译模板，如下所示：

```

1  // 需要编译器的版本
2  new Vue({
3    template: '<div>{{ hi }}</div>'
4  })
5
6  // 这种情况不需要
7  new Vue({
8    render (h) {
9      return h('div', this.hi)
10   }
11 })

```

在 Vue.js 2.0 中，最终渲染都是通过 render 函数，如果写 template 属性，则需要编译成 render 函数，那么这个编译过程会发生运行时，所以需要带有编译器的版本。

很显然，这个编译过程对性能会有一定损耗，所以通常我们更推荐使用 Runtime-Only 的 Vue.js。

Vue入口

在 web 场景下，我们来分析 Runtime + Compiler 构建出来的 Vue.js，它的入口是

`src/platforms/web/entry-runtime-with-compiler.js`：

```

1  /* @flow */
2
3  import config from 'core/config'
4  import { warn, cached } from 'core/util/index'
5  import { mark, measure } from 'core/util/perf'
6
7  import Vue from './runtime/index'
8  import { query } from './util/index'
9  import { compileToFunctions } from './compiler/index'
10 import { shouldDecodeNewlines, shouldDecodeNewlinesForHref } from
    './util/compat'
11
12 const idToTemplate = cached(id => {
13   const el = query(id)
14   return el && el.innerHTML
15 })
16
17 const mount = Vue.prototype.$mount
18 Vue.prototype.$mount = function (
19   el?: string | Element,
20   hydrating?: boolean
21 ): Component {

```

```
22     el = el && query(el)
23
24     /* istanbul ignore if */
25     if (el === document.body || el === document.documentElement) {
26         process.env.NODE_ENV !== 'production' && warn(
27             `Do not mount Vue to <html> or <body> - mount to normal elements
instead.`
28         )
29         return this
30     }
31
32     const options = this.$options
33     // resolve template/el and convert to render function
34     if (!options.render) {
35         let template = options.template
36         if (template) {
37             if (typeof template === 'string') {
38                 if (template.charAt(0) === '#') {
39                     template = idToTemplate(template)
40                     /* istanbul ignore if */
41                     if (process.env.NODE_ENV !== 'production' && !template) {
42                         warn(
43                             `Template element not found or is empty: ${options.template}`,
44                             this
45                         )
46                     }
47                 }
48             } else if (template.nodeType) {
49                 template = template.innerHTML
50             } else {
51                 if (process.env.NODE_ENV !== 'production') {
52                     warn('invalid template option:' + template, this)
53                 }
54                 return this
55             }
56         } else if (el) {
57             template = getOuterHTML(el)
58         }
59         if (template) {
60             /* istanbul ignore if */
61             if (process.env.NODE_ENV !== 'production' && config.performance &&
mark) {
62                 mark('compile')
63             }
64
65             const { render, staticRenderFns } = compileToFunctions(template, {
66                 outputSourceRange: process.env.NODE_ENV !== 'production',
```

```

67         shouldDecodeNewlines,
68         shouldDecodeNewlinesForHref,
69         delimiters: options.delimiters,
70         comments: options.comments
71     }, this)
72     options.render = render
73     options.staticRenderFns = staticRenderFns
74
75     /* istanbul ignore if */
76     if (process.env.NODE_ENV !== 'production' && config.performance &&
mark) {
77         mark('compile end')
78         measure(`vue ${this._name} compile`, 'compile', 'compile end')
79     }
80 }
81 }
82 return mount.call(this, el, hydrating)
83 }
84
85 /**
86  * Get outerHTML of elements, taking care
87  * of SVG elements in IE as well.
88  */
89 function getOuterHTML (el: Element): string {
90     if (el.outerHTML) {
91         return el.outerHTML
92     } else {
93         const container = document.createElement('div')
94         container.appendChild(el.cloneNode(true))
95         return container.innerHTML
96     }
97 }
98
99 Vue.compile = compileToFunctions
100
101 export default Vue
102

```

当我们的代码执行 `import Vue from 'vue'` 的时候，就是从这个入口执行代码来初始化 Vue，来源为：

```

import Vue from './runtime/index'，入口在
src/platforms/web/runtime/index.js

```

```

1  /* @flow */
2

```

```
3 import Vue from 'core/index'
4 import config from 'core/config'
5 import { extend, noop } from 'shared/util'
6 import { mountComponent } from 'core/instance/lifecycle'
7 import { devtools, inBrowser } from 'core/util/index'
8
9 import {
10   query,
11   mustUseProp,
12   isReservedTag,
13   isReservedAttr,
14   getTagNamespace,
15   isUnknownElement
16 } from 'web/util/index'
17
18 import { patch } from './patch'
19 import platformDirectives from './directives/index'
20 import platformComponents from './components/index'
21
22 // install platform specific utils
23 Vue.config.mustUseProp = mustUseProp
24 Vue.config.isReservedTag = isReservedTag
25 Vue.config.isReservedAttr = isReservedAttr
26 Vue.config.getTagNamespace = getTagNamespace
27 Vue.config.isUnknownElement = isUnknownElement
28
29 // install platform runtime directives & components
30 extend(Vue.options.directives, platformDirectives)
31 extend(Vue.options.components, platformComponents)
32
33 // install platform patch function
34 Vue.prototype.__patch__ = inBrowser ? patch : noop
35
36 // public mount method
37 Vue.prototype.$mount = function (
38   el?: string | Element,
39   hydrating?: boolean
40 ): Component {
41   el = el && inBrowser ? query(el) : undefined
42   return mountComponent(this, el, hydrating)
43 }
44
45 // devtools global hook
46 /* istanbul ignore next */
47 if (inBrowser) {
48   setTimeout(() => {
49     if (config.devtools) {
```

```

50     if (devtools) {
51       devtools.emit('init', Vue)
52     } else if (
53       process.env.NODE_ENV !== 'production' &&
54       process.env.NODE_ENV !== 'test'
55     ) {
56       console[console.info ? 'info' : 'log'](
57         'Download the Vue Devtools extension for a better development
experience:\n' +
58         'https://github.com/vuejs/vue-devtools'
59       )
60     }
61   }
62   if (process.env.NODE_ENV !== 'production' &&
63     process.env.NODE_ENV !== 'test' &&
64     config.productionTip !== false &&
65     typeof console !== 'undefined'
66   ) {
67     console[console.info ? 'info' : 'log'](
68       `You are running Vue in development mode.\n` +
69       `Make sure to turn on production mode when deploying for
production.\n` +
70       `See more tips at https://vuejs.org/guide/deployment.html`
71     )
72   }
73 }, 0)
74 }
75
76 export default Vue
77

```

里关键的代码是 `import Vue from 'core/index'`，之后的逻辑都是对 Vue 这个对象做一些扩展，可以先不用看，我们来看一下真正初始化 Vue 的地方，在 `src/core/index.js` 中：

```

1  import Vue from './instance/index'
2  import { initGlobalAPI } from './global-api/index'
3  import { isServerRendering } from 'core/util/env'
4  import { FunctionalRenderContext } from 'core/vdom/create-functional-
component'
5
6  initGlobalAPI(Vue)
7
8  Object.defineProperty(Vue.prototype, '$isServer', {
9    get: isServerRendering
10 })

```

```

11
12 Object.defineProperty(Vue.prototype, '$ssrContext', {
13   get () {
14     /* istanbul ignore next */
15     return this.$vnode && this.$vnode.ssrContext
16   }
17 })
18
19 // expose FunctionalRenderContext for ssr runtime helper installation
20 Object.defineProperty(Vue, 'FunctionalRenderContext', {
21   value: FunctionalRenderContext
22 })
23
24 Vue.version = '__VERSION__'
25
26 export default Vue

```

这里有 2 处关键的代码，`import Vue from './instance/index'` 和 `initGlobalAPI(Vue)`，初始化全局 Vue API（我们稍后介绍），我们先来看第一部分，在 `src/core/instance/index.js` 中：

Vue定义

```

1  import { initMixin } from './init'
2  import { stateMixin } from './state'
3  import { renderMixin } from './render'
4  import { eventsMixin } from './events'
5  import { lifecycleMixin } from './lifecycle'
6  import { warn } from '../util/index'
7
8  function Vue (options) {
9    if (process.env.NODE_ENV !== 'production' &&
10      !(this instanceof Vue)
11    ) {
12      warn('Vue is a constructor and should be called with the `new` keyword')
13    }
14    this._init(options)
15  }
16
17  initMixin(Vue)
18  stateMixin(Vue)
19  eventsMixin(Vue)
20  lifecycleMixin(Vue)
21  renderMixin(Vue)
22

```

在这里，我们会发现，Vue实际上就是一个用 Function 实现的类，这也是为什么我们只能通过 `new Vue` 去实例化它的原因。

Q：为何 Vue 不用 ES6 的 Class 去实现呢？

A：我们往后看这里有很多 xxxMixin 的函数调用，并把 Vue 当参数传入，它们的功能都是给 Vue 的 prototype 上扩展一些方法，Vue 按功能把这些扩展分散到多个模块中去实现，而不是在一个模块里实现所有，这种方式是用 Class 难以实现的。这么做的好处是非常方便代码的维护和管理。

initGlobalAPI

Vue.js 在整个初始化过程中，除了给它的原型 prototype 上扩展方法，还会给 Vue 这个对象本身扩展全局的静态方法，它的定义在 `src/core/global-api/index.js` 中：

```
1  export function initGlobalAPI (Vue: GlobalAPI) {
2    // config
3    const configDef = {}
4    configDef.get = () => config
5    if (process.env.NODE_ENV !== 'production') {
6      configDef.set = () => {
7        warn(
8          'Do not replace the Vue.config object, set individual fields instead.'
9        )
10     }
11   }
12   Object.defineProperty(Vue, 'config', configDef)
13
14   // exposed util methods.
15   // NOTE: these are not considered part of the public API - avoid relying on
16   // them unless you are aware of the risk.
17   Vue.util = {
18     warn,
19     extend,
20     mergeOptions,
21     defineReactive
22   }
23
24   Vue.set = set
25   Vue.delete = del
26   Vue.nextTick = nextTick
27 }
```

```

28   Vue.options = Object.create(null)
29   ASSET_TYPES.forEach(type => {
30     Vue.options[type + 's'] = Object.create(null)
31   })
32
33   // this is used to identify the "base" constructor to extend all plain-
  object
34   // components with in Weex's multi-instance scenarios.
35   Vue.options._base = Vue
36
37   extend(Vue.options.components, builtInComponents)
38
39   initUse(Vue)
40   initMixin(Vue)
41   initExtend(Vue)
42   initAssetRegisters(Vue)
43 }

```

这里就是在 Vue 上扩展的一些全局方法的定义，Vue 官网中关于全局 API 都可以在这里找到，有一点要注意的是，`Vue.util` 暴露的方法最好不要依赖，因为它可能经常会发生变化，是不稳定的。具体的全局API后面详细讲解；

数据驱动

Vue.js 一个核心思想是数据驱动。所谓数据驱动，是指视图是由数据驱动生成的，我们对视图的修改，不会直接操作 DOM，而是通过修改数据。它相比我们传统的前端开发，如使用 jQuery 等前端库直接修改 DOM，大大简化了代码量。特别是当交互复杂的时候，只关心数据的修改会让代码的逻辑变的非常清晰，因为 DOM 变成了数据的映射，我们所有的逻辑都是对数据的修改，而不用碰触 DOM，这样的代码非常利于维护。

在 Vue.js 中我们可以采用简洁的模板语法来声明式的将数据渲染为 DOM：

```

1   <div id="app">
2     {{ message }}
3   </div>
4
5   var app = new Vue({
6     el: '#app',
7     data: {
8       message: 'Hello Vue!'
9     }
10  })

```

最终它会在页面上渲染出 `Hello Vue`。接下来，我们会从源码角度来分析 Vue 是如何实现的。在这里，先弄清楚模板和数据如何渲染成最终的 DOM。

new Vue 时发生了什么

从入口代码开始分析，我们先来分析 new Vue 背后发生了哪些事情。我们都知道，new 关键字在 Javascript 语言中代表实例化是一个对象，而 Vue 实际上是一个类，类在 Javascript 中是用 Function 来实现的，来看一下源码，在 `src/core/instance/index.js` 中。

```
1 function Vue (options) {
2   if (process.env.NODE_ENV !== 'production' &&
3     !(this instanceof Vue)
4   ) {
5     warn('Vue is a constructor and should be called with the `new` keyword')
6   }
7   this._init(options)
8 }
```

可以看到 Vue 只能通过 new 关键字初始化，然后会调用 this._init 方法，该方法在 `initMixin(Vue)` 时注入到 prototype 上，在 `src/core/instance/init.js` 中定义。

```
1 Vue.prototype._init = function (options?: Object) {
2   const vm: Component = this
3   // a uid
4   vm._uid = uid++
5
6   let startTag, endTag
7   /* istanbul ignore if */
8   if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
9     startTag = `vue-perf-start:${vm._uid}`
10    endTag = `vue-perf-end:${vm._uid}`
11    mark(startTag)
12  }
13
14  // a flag to avoid this being observed
15  vm._isVue = true
16  // merge options
17  if (options && options._isComponent) {
18    // optimize internal component instantiation
19    // since dynamic options merging is pretty slow, and none of the
20    // internal component options needs special treatment.
21    initInternalComponent(vm, options)
22  } else {
23    vm.$options = mergeOptions(
```

```

24     resolveConstructorOptions(vm.constructor),
25     options || {},
26     vm
27   )
28 }
29 /* istanbul ignore else */
30 if (process.env.NODE_ENV !== 'production') {
31   initProxy(vm)
32 } else {
33   vm._renderProxy = vm
34 }
35 // expose real self
36 vm._self = vm
37 initLifecycle(vm)
38 initEvents(vm)
39 initRender(vm)
40 callHook(vm, 'beforeCreate')
41 initInjections(vm) // resolve injections before data/props
42 initState(vm)
43 initProvide(vm) // resolve provide after data/props
44 callHook(vm, 'created')
45
46 /* istanbul ignore if */
47 if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
48   vm._name = formatComponentName(vm, false)
49   mark(endTag)
50   measure(`vue ${vm._name} init`, startTag, endTag)
51 }
52
53 if (vm.$options.el) {
54   vm.$mount(vm.$options.el)
55 }
56 }

```

Vue 初始化主要就干了几件事情，合并配置，初始化生命周期，初始化事件中心，初始化渲染，初始化 data、props、computed、watcher 等等。

Vue 的初始化逻辑写的非常清楚，把不同的功能逻辑拆成一些单独的函数执行，让主线逻辑一目了然，由于我们这一章的目标是弄清楚模板和数据如何渲染成最终的 DOM，所以各种初始化逻辑我们先不看。在初始化的最后，检测到如果有 el 属性，则调用 vm.\$mount 方法挂载 vm，挂载的目标就是把模板渲染成最终的 DOM，那么接下来我们分析 Vue 的挂载过程。

Vue实例挂载的实现

Vue 中我们是通过 `$mount` 实例方法去挂载 `vm` 的，`$mount` 方法在多个文件中都有定义，如 `src/platform/web/entry-runtime-with-compiler.js`、`src/platform/web/runtime/index.js`、`src/platform/weex/runtime/index.js`。因为 `$mount` 这个方法的实现是和平台、构建方式都相关的。接下来我们重点分析带 `compiler` 版本的 `$mount` 实现，因为抛开 `webpack` 的 `vue-loader`，我们在纯前端浏览器环境分析 `Vue` 的工作原理，有助于我们对原理理解的深入。

先来看一下 `src/platform/web/entry-runtime-with-compiler.js` 文件中定义：

```
1  const mount = Vue.prototype.$mount
2  Vue.prototype.$mount = function (
3    el?: string | Element,
4    hydrating?: boolean
5  ): Component {
6    el = el && query(el)
7
8    /* istanbul ignore if */
9    if (el === document.body || el === document.documentElement) {
10      process.env.NODE_ENV !== 'production' && warn(
11        `Do not mount Vue to <html> or <body> - mount to normal elements
12        instead.`
13      )
14      return this
15    }
16
17    const options = this.$options
18    // resolve template/el and convert to render function
19    if (!options.render) {
20      let template = options.template
21      if (template) {
22        if (typeof template === 'string') {
23          if (template.charAt(0) === '#') {
24            template = idToTemplate(template)
25            /* istanbul ignore if */
26            if (process.env.NODE_ENV !== 'production' && !template) {
27              warn(
28                `Template element not found or is empty: ${options.template}`,
29                this
30              )
31            }
32          } else if (template.nodeType) {
33            template = template.innerHTML
34          } else {
35            if (process.env.NODE_ENV !== 'production') {
36              warn('invalid template option:' + template, this)
```

```

37     }
38     return this
39   }
40   } else if (el) {
41     template = getOuterHTML(el)
42   }
43   if (template) {
44     /* istanbul ignore if */
45     if (process.env.NODE_ENV !== 'production' && config.performance &&
mark) {
46       mark('compile')
47     }
48
49     const { render, staticRenderFns } = compileToFunctions(template, {
50       outputSourceRange: process.env.NODE_ENV !== 'production',
51       shouldDecodeNewlines,
52       shouldDecodeNewlinesForHref,
53       delimiters: options.delimiters,
54       comments: options.comments
55     }, this)
56     options.render = render
57     options.staticRenderFns = staticRenderFns
58
59     /* istanbul ignore if */
60     if (process.env.NODE_ENV !== 'production' && config.performance &&
mark) {
61       mark('compile end')
62       measure(`vue ${this._name} compile`, 'compile', 'compile end')
63     }
64   }
65 }
66 return mount.call(this, el, hydrating)
67 }

```

这段代码首先缓存了原型上的 `$mount` 方法，再重新定义该方法

1. 它对 `el` 做了限制，Vue 不能挂载在 `body`、`html` 这样的根节点上；
2. 如果没有定义 `render` 方法，则会把 `el` 或者 `template` 字符串转换成 `render` 方法。在 Vue 2.0 版本中，所有 Vue 的组件的渲染最终都需要 `render` 方法，无论我们是用单文件 `.vue` 方式开发组件，还是写了 `el` 或者 `template` 属性，最终都会转换成 `render` 方法；
3. 根据生成的 `template` 函数，会执行在线编译的过程，是调用 `compileToFunctions` 方法实现的，在后续的编译过程中讲解。最后，调用原先原型上的 `$mount` 方法挂载；

原先原型上的 `$mount` 方法在 `src/platform/web/runtime/index.js` 中定义，之所以这么设计完全是为了复用，因为它是可以被 `runtime only` 版本的 Vue 直接使用的。

```
1  // public mount method
2  Vue.prototype.$mount = function (
3    el?: string | Element,
4    hydrating?: boolean
5  ): Component {
6    el = el && inBrowser ? query(el) : undefined
7    return mountComponent(this, el, hydrating)
8  }
```

`$mount` 方法支持传入 2 个参数，第一个是 `el`，它表示挂载的元素，可以是字符串，也可以是 DOM 对象，如果是字符串在浏览器环境下会调用 `query` 方法转换成 DOM 对象的。第二个参数是和服务端渲染相关，在浏览器环境下我们不需要传第二个参数。

`$mount` 方法实际上会去调用 `mountComponent` 方法，这个方法定义在 `src/core/instance/lifecycle.js` 文件中：

```
1  export function mountComponent (
2    vm: Component,
3    el: ?Element,
4    hydrating?: boolean
5  ): Component {
6    vm.$el = el
7    if (!vm.$options.render) {
8      vm.$options.render = createEmptyVNode
9      if (process.env.NODE_ENV !== 'production') {
10        /* istanbul ignore if */
11        if ((vm.$options.template && vm.$options.template.charAt(0) !== '#') ||
12          vm.$options.el || el) {
13          warn(
14            'You are using the runtime-only build of Vue where the template ' +
15            'compiler is not available. Either pre-compile the templates into '
16            +
17            'render functions, or use the compiler-included build.',
18            vm
19          )
20        } else {
21          warn(
22            'Failed to mount component: template or render function not
23            defined.',
```

```
22         vm
23     )
24 }
25 }
26 }
27 callHook(vm, 'beforeMount')
28
29 let updateComponent
30 /* istanbul ignore if */
31 if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
32     updateComponent = () => {
33         const name = vm._name
34         const id = vm._uid
35         const startTag = `vue-perf-start:${id}`
36         const endTag = `vue-perf-end:${id}`
37
38         mark(startTag)
39         const vnode = vm._render()
40         mark(endTag)
41         measure(`vue ${name} render`, startTag, endTag)
42
43         mark(startTag)
44         vm._update(vnode, hydrating)
45         mark(endTag)
46         measure(`vue ${name} patch`, startTag, endTag)
47     }
48 } else {
49     updateComponent = () => {
50         vm._update(vm._render(), hydrating)
51     }
52 }
53
54 // we set this to vm._watcher inside the watcher's constructor
55 // since the watcher's initial patch may call $forceUpdate (e.g. inside
child
56 // component's mounted hook), which relies on vm._watcher being already
defined
57 new Watcher(vm, updateComponent, noop, {
58     before () {
59         if (vm._isMounted) {
60             callHook(vm, 'beforeUpdate')
61         }
62     }
63 }, true /* isRenderWatcher */)
64 hydrating = false
65
66 // manually mounted instance, call mounted on self
```

```

67   // mounted is called for render-created child components in its inserted
    hook
68   if (vm.$vnode == null) {
69     vm._isMounted = true
70     callHook(vm, 'mounted')
71   }
72   return vm
73 }

```

这里面核心就是先实例化一个渲染Watcher，在它的回调函数中会调用 `updateComponent` 方法，在此方法中调用 `vm._render` 方法先生成虚拟 Node，最终调用 `vm._update` 更新 DOM。

Watcher 在这里起到两个作用：（后面会详细讲解）

1. 初始化的时候会执行回调函数；
2. 当 vm 实例中的监测的数据发生变化的时候执行回调函数；

函数最后判断当根节点 `vm.$vnode` 为 null 时，执行 mount 初始化；接下来详细讲 `_render`（生成 VNode）和 `_update`（更新 DOM）

render

Vue 的 `_render` 方法是实例的一个私有方法，它用来把实例渲染成一个虚拟 Node。它的定义在 `src/core/instance/render.js` 文件中：

在 `src/core/instance/init.js` 中，执行 `renderMixin`，注入到到 `Vue.prototype` 上：

```

1  Vue.prototype._render = function (): VNode {
2    const vm: Component = this
3    const { render, _parentVnode } = vm.$options
4
5    // reset _rendered flag on slots for duplicate slot check
6    if (process.env.NODE_ENV !== 'production') {
7      for (const key in vm.$slots) {
8        // $flow-disable-line
9        vm.$slots[key]._rendered = false
10     }
11   }
12
13   if (_parentVnode) {
14     vm.$scopedSlots = _parentVnode.data.scopedSlots || emptyObject
15   }
16
17   // set parent vnode. this allows render functions to have access
18   // to the data on the placeholder node.

```

```

19   vm.$vnode = _parentVnode
20   // render self
21   let vnode
22   try {
23     vnode = render.call(vm._renderProxy, vm.$createElement)
24   } catch (e) {
25     handleError(e, vm, `render`)
26     // return error render result,
27     // or previous vnode to prevent render error causing blank component
28     /* istanbul ignore else */
29     if (process.env.NODE_ENV !== 'production') {
30       if (vm.$options.renderError) {
31         try {
32           vnode = vm.$options.renderError.call(vm._renderProxy,
vm.$createElement, e)
33         } catch (e) {
34           handleError(e, vm, `renderError`)
35           vnode = vm._vnode
36         }
37       } else {
38         vnode = vm._vnode
39       }
40     } else {
41       vnode = vm._vnode
42     }
43   }
44   // return empty vnode in case the render function errored out
45   if (!(vnode instanceof VNode)) {
46     if (process.env.NODE_ENV !== 'production' && Array.isArray(vnode)) {
47       warn(
48         'Multiple root nodes returned from render function. Render function '
+
49         'should return a single root node.',
50         vm
51       )
52     }
53     vnode = createEmptyVNode()
54   }
55   // set parent
56   vnode.parent = _parentVnode
57   return vnode
58 }

```

最关键的是 render 方法的调用，在之前的 mounted 方法的实现中，会把 template 编译成 render 方法，具体的编译方法在后面讲

在_render 函数中的 render 方法的调用中：

```
1  vnode = render.call(vm._renderProxy, vm.$createElement)
```

render 函数中的 createElement 方法就是 vm.\$createElement 方法，其中vm.\$createElement的方法是在initRender中定义的，其中vm.\$createElement调用了createElement，另一个方法也调用了，这个方法是模板编译成的 render 函数使用，而 vm.\$createElement 是用户手写 render 方法使用的，这两个方法支持的参数相同，并且内部都调用了 createElement 方法

```
1  vm._c = (a, b, c, d) => createElement(vm, a, b, c, d, false)
2  vm.$createElement = (a, b, c, d) => createElement(vm, a, b, c, d, true)
```

此方法，也是Vue中提到的render的第一个参数 createElement

```
1  render: function (createElement) {
2    return createElement('div', {
3      attrs: {
4        id: 'app'
5      },
6    }, this.message)
7  }
```

VDOM

在了解createElement原理之前，先了解下Virtual DOM是什么。

在浏览器中，一个最简单的div所包含的元素也是很多的，因为浏览器的标准就把 DOM 设计的非常复杂。当我们频繁的去做 DOM 更新，会产生一定的性能问题。

```
1  const div = document.createElement('div')
2  let str = '';
3  for (const key in div) str += key + ' '
```

而 Virtual DOM 就是用一个原生的 JS 对象去描述一个 DOM 节点，所以它比创建一个 DOM 的代价要小很多。在 Vue.js 中，Virtual DOM 是用 VNode 这么一个 Class 去描述，它是定义在 `src/core/vdom/vnode.js` 中的。

```

1  export default class VNode {
2      tag: string | void;
3      data: VNodeData | void;
4      children: ?Array<VNode>;
5      text: string | void;
6      elm: Node | void;
7      ns: string | void;
8      context: Component | void; // rendered in this component's scope
9      key: string | number | void;
10     componentOptions: VNodeComponentOptions | void;
11     componentInstance: Component | void; // component instance
12     parent: VNode | void; // component placeholder node
13
14     // strictly internal
15     raw: boolean; // contains raw HTML? (server only)
16     isStatic: boolean; // hoisted static node
17     isRootInsert: boolean; // necessary for enter transition check
18     isComment: boolean; // empty comment placeholder?
19     isCloned: boolean; // is a cloned node?
20     isOnce: boolean; // is a v-once node?
21     asyncFactory: Function | void; // async component factory function
22     asyncMeta: Object | void;
23     isAsyncPlaceholder: boolean;
24     ssrContext: Object | void;
25     fnContext: Component | void; // real context vm for functional nodes
26     fnOptions: ?ComponentOptions; // for SSR caching
27     fnScopeId: ?string; // functional scope id support
28
29     constructor (
30         tag?: string,
31         data?: VNodeData,
32         children?: ?Array<VNode>,
33         text?: string,
34         elm?: Node,
35         context?: Component,
36         componentOptions?: VNodeComponentOptions,
37         asyncFactory?: Function
38     ) {
39         this.tag = tag
40         this.data = data
41         this.children = children
42         this.text = text
43         this.elm = elm
44         this.ns = undefined
45         this.context = context
46         this.fnContext = undefined
47         this.fnOptions = undefined

```

```

48     this.fnScopeId = undefined
49     this.key = data && data.key
50     this.componentOptions = componentOptions
51     this.componentInstance = undefined
52     this.parent = undefined
53     this.raw = false
54     this.isStatic = false
55     this.isRootInsert = true
56     this.isComment = false
57     this.isCloned = false
58     this.isOnce = false
59     this.asyncFactory = asyncFactory
60     this.asyncMeta = undefined
61     this.isAsyncPlaceholder = false
62   }
63
64   // DEPRECATED: alias for componentInstance for backwards compat.
65   /* istanbul ignore next */
66   get child (): Component | void {
67     return this.componentInstance
68   }
69 }

```

实际上 Vue.js 中 Virtual DOM 是借鉴了一个开源库 [snabbdom](#) 的实现，然后加入了一些 Vue.js 本身的东西，在后面的内容里，会在 VNode 的 create、diff、patch 等阶段讲解 VNode 的操作；

createElement

Vue.js 利用 createElement 方法创建 VNode，它定义在 `src/core/vdom/create-element.js` 中：

```

1  // wrapper function for providing a more flexible interface
2  // without getting yelled at by flow
3  export function createElement (
4    context: Component,
5    tag: any,
6    data: any,
7    children: any,
8    normalizationType: any,
9    alwaysNormalize: boolean
10 ): VNode | Array<VNode> {
11   if (Array.isArray(data) || isPrimitive(data)) {
12     normalizationType = children
13     children = data

```

```

14     data = undefined
15   }
16   if (isTrue(alwaysNormalize)) {
17     normalizationType = ALWAYS_NORMALIZE
18   }
19   return _createElement(context, tag, data, children, normalizationType)
20 }
21
22 export function _createElement (
23   context: Component,
24   tag?: string | Class<Component> | Function | Object,
25   data?: VNodeData,
26   children?: any,
27   normalizationType?: number
28 ): VNode | Array<VNode> {
29   if (isDef(data) && isDef((data: any).__ob__)) {
30     process.env.NODE_ENV !== 'production' && warn(
31       `Avoid using observed data object as vnode data:
32       ${JSON.stringify(data)}\n` +
33       'Always create fresh vnode data objects in each render!',
34       context
35     )
36     return createEmptyVNode()
37   }
38   // object syntax in v-bind
39   if (isDef(data) && isDef(data.is)) {
40     tag = data.is
41   }
42   if (!tag) {
43     // in case of component :is set to falsy value
44     return createEmptyVNode()
45   }
46   // warn against non-primitive key
47   if (process.env.NODE_ENV !== 'production' &&
48     isDef(data) && isDef(data.key) && !isPrimitive(data.key)
49   ) {
50     if (!__WEEX__ || !('@binding' in data.key)) {
51       warn(
52         'Avoid using non-primitive value as key, ' +
53         'use string/number value instead.',
54         context
55       )
56     }
57   }
58   // support single function children as default scoped slot
59   if (Array.isArray(children) &&
60     typeof children[0] === 'function'

```

```

60   ) {
61     data = data || {}
62     data.scopedSlots = { default: children[0] }
63     children.length = 0
64   }
65   if (normalizationType === ALWAYS_NORMALIZE) {
66     children = normalizeChildren(children)
67   } else if (normalizationType === SIMPLE_NORMALIZE) {
68     children = simpleNormalizeChildren(children)
69   }
70   let vnode, ns
71   if (typeof tag === 'string') {
72     let Ctor
73     ns = (context.$vnode && context.$vnode.ns) || config.getTagNamespace(tag)
74     if (config.isReservedTag(tag)) {
75       // platform built-in elements
76       if (process.env.NODE_ENV !== 'production' && isDef(data) &&
isDef(data.nativeOn) && data.tag !== 'component') {
77         warn(
78           `The .native modifier for v-on is only valid on components but it
was used on <${tag}>`,
79           context
80         )
81       }
82       vnode = new VNode(
83         config.parsePlatformTagName(tag), data, children,
84         undefined, undefined, context
85       )
86     } else if ((!data || !data.pre) && isDef(Ctor =
resolveAsset(context.$options, 'components', tag))) {
87       // component
88       vnode = createComponent(Ctor, data, context, children, tag)
89     } else {
90       // unknown or unlisted namespaced elements
91       // check at runtime because it may get assigned a namespace when its
92       // parent normalizes children
93       vnode = new VNode(
94         tag, data, children,
95         undefined, undefined, context
96       )
97     }
98   } else {
99     // direct component options / constructor
100    vnode = createComponent(tag, data, context, children)
101  }
102  if (Array.isArray(vnode)) {
103    return vnode

```

```

104     } else if (isDef(vnode)) {
105         if (isDef(ns)) applyNS(vnode, ns)
106         if (isDef(data)) registerDeepBindings(data)
107         return vnode
108     } else {
109         return createEmptyVNode()
110     }
111 }
112

```

`_createElement` 方法有 5 个参数：

1. context 表示 VNode 的上下文环境，它是 Component 类型；
2. tag 表示标签，它可以是一个字符串，也可以是一个 Component；
3. data 表示 VNode 的数据，它是一个 VNodeData 类型，可以在 `flow/vnode.js` 中找到它的定义，这里先不展开说；
4. children 表示当前 VNode 的子节点，它是任意类型的，它接下来需要被规范为标准的 VNode 数组；
5. normalizationType 表示子节点规范的类型，类型不同规范的方法也就不一样，它主要是参考 `render` 函数是编译生成的还是用户手写的。

`createElement` 里核心的流程包括两个：

children 的规范化

由于 Virtual DOM 实际上是一个树状结构，每一个 VNode 可能会有若干个子节点，这些子节点应该也是 VNode 的类型。`_createElement` 接收的第 4 个参数 `children` 是任意类型的，因此我们需要把它们规范成 VNode 类型。

这里根据 `normalizationType` 的不同，调用了 `normalizeChildren(children)` 和 `simpleNormalizeChildren(children)` 方法，它们的定义都在 `src/core/vdom/helpers/normalize-children.js` 中：

```

1  export function simpleNormalizeChildren (children: any) {
2      for (let i = 0; i < children.length; i++) {
3          if (Array.isArray(children[i])) {
4              return Array.prototype.concat.apply([], children)
5          }
6      }
7      return children
8  }

```

```

9
10 export function normalizeChildren (children: any): ?Array<VNode> {
11   return isPrimitive(children)
12     ? [createTextVNode(children)]
13     : Array.isArray(children)
14       ? normalizeArrayChildren(children)
15       : undefined
16 }

```

- simpleNormalizeChildren:

1. 调用场景是 render 函数是编译生成的。
2. 理论上编译生成的 children 都已经是 VNode 类型的，但这里有一个例外，就是 functional component 函数式组件返回的是一个数组而不是一个根节点，所以会通过 `Array.prototype.concat` 方法把整个 children 数组打平，让它的深度只有一层。

- normalizeChildren 方法的调用场景有 2 种：

1. 一个场景是 render 函数是用户手写的，当 children 只有一个节点的时候，Vue.js 从接口层面允许用户把 children 写成基础类型用来创建单个简单的文本节点，这种情况会调用 `createTextVNode` 创建一个文本节点的 VNode；
2. 另一个场景是当编译 slot、v-for 的时候会产生嵌套数组的情况，会调用 `normalizeArrayChildren` 方法；

```

1  function normalizeArrayChildren (children: any, nestedIndex?: string):
    Array<VNode> {
2    const res = []
3    let i, c, lastIndex, last
4    for (i = 0; i < children.length; i++) {
5      c = children[i]
6      if (isUndef(c) || typeof c === 'boolean') continue
7      lastIndex = res.length - 1
8      last = res[lastIndex]
9      // nested
10     if (Array.isArray(c)) {
11       if (c.length > 0) {
12         c = normalizeArrayChildren(c, `${nestedIndex} || '$_${i}'`)
13         // merge adjacent text nodes
14         if (isTextNode(c[0]) && isTextNode(last)) {
15           res[lastIndex] = createTextVNode(last.text + (c[0]: any).text)
16           c.shift()
17         }
18         res.push.apply(res, c)
19       }

```

```

20     } else if (isPrimitive(c)) {
21       if (isTextNode(last)) {
22         // merge adjacent text nodes
23         // this is necessary for SSR hydration because text nodes are
24         // essentially merged when rendered to HTML strings
25         res[lastIndex] = createTextVNode(last.text + c)
26       } else if (c !== '') {
27         // convert primitive to vnode
28         res.push(createTextVNode(c))
29       }
30     } else {
31       if (isTextNode(c) && isTextNode(last)) {
32         // merge adjacent text nodes
33         res[lastIndex] = createTextVNode(last.text + c.text)
34       } else {
35         // default key for nested array children (likely generated by v-for)
36         if (isTrue(children._isVList) &&
37             isDef(c.tag) &&
38             isUndef(c.key) &&
39             isDef(nestedIndex)) {
40           c.key = `__vlist${nestedIndex}_${i}__`
41         }
42         res.push(c)
43       }
44     }
45   }
46   return res
47 }
48

```

normalizeArrayChildren 接收 2 个参数：

1. children：表示要规范的子节点；
2. nestedIndex：表示嵌套的索引；

因为单个 child 可能是一个数组类型。normalizeArrayChildren 主要的逻辑就是遍历 children，获得单个节点 c，然后对 c 的类型判断：

1. 如果是一个数组类型，则递归调用 normalizeArrayChildren；
2. 如果是基础类型，则通过 createTextVNode 方法转换成 VNode 类型；否则就已经是 VNode 类型了，如果 children 是一个列表并且列表还存在嵌套的情况，则根据 nestedIndex 去更新它的 key；

注意：在遍历的过程中，如果存在两个连续的 text 节点，会把它们合并成一个 text 节点

然后，children就变为了VNode的单节点或者数组了

VNode的构建

规范完children后，需要创建一个VNode实例：

```
1  let vnode, ns
2  if (typeof tag === 'string') {
3    let Ctor
4    ns = (context.$vnode && context.$vnode.ns) || config.getTagNamespace(tag)
5    if (config.isReservedTag(tag)) {
6      // platform built-in elements
7      if (process.env.NODE_ENV !== 'production' && isDef(data) &&
isDef(data.nativeOn) && data.tag !== 'component') {
8        warn(
9          `The .native modifier for v-on is only valid on components but it was
used on <${tag}>.` ,
10         context
11       )
12     }
13     vnode = new VNode(
14       config.parsePlatformTagName(tag), data, children,
15       undefined, undefined, context
16     )
17   } else if ((!data || !data.pre) && isDef(Ctor =
resolveAsset(context.$options, 'components', tag))) {
18     // component
19     vnode = createComponent(Ctor, data, context, children, tag)
20   } else {
21     // unknown or unlisted namespaced elements
22     // check at runtime because it may get assigned a namespace when its
23     // parent normalizes children
24     vnode = new VNode(
25       tag, data, children,
26       undefined, undefined, context
27     )
28   }
29 } else {
30   // direct component options / constructor
31   vnode = createComponent(tag, data, context, children)
32 }
```

这里先对 tag 做判断：

1. 如果是 string 类型，则接着判断如果是内置的一些节点，则直接创建一个普通 VNode；

2. 如果是为已注册的组件名，则通过 `createComponent` 创建一个组件类型的 VNode，否则创建一个未知的标签的 VNode；
3. 如果是 tag 一个 Component 类型，则直接调用 `createComponent` 创建一个组件类型的 VNode 节点。对于 `createComponent` 创建组件类型的 VNode 的过程，本质上它还是返回了一个 VNode，后面讲解。

因此，createElement 创建 VNode 的过程，每个 VNode 有 children，children 每个元素也是一个 VNode，这样就形成了一个 VNode Tree，它很好的描述了我们的 DOM Tree。

update

Vue 的 `_update` 是实例的一个私有方法，它被调用的时机有 2 个，一个是首次渲染，一个是数据更新的时候；（涉及到数据更新的代码会在后面讲到），`_update` 方法的作用是把 VNode 渲染成真实的 DOM，它的定义在 `src/core/instance/lifecycle.js` 中：

```
1  Vue.prototype._update = function (vnode: VNode, hydrating?: boolean) {
2    const vm: Component = this
3    const prevEl = vm.$el
4    const prevVnode = vm._vnode
5    const prevActiveInstance = activeInstance
6    activeInstance = vm
7    vm._vnode = vnode
8    // Vue.prototype.__patch__ is injected in entry points
9    // based on the rendering backend used.
10   if (!prevVnode) {
11     // initial render
12     vm.$el = vm.__patch__(vm.$el, vnode, hydrating, false /* removeOnly */)
13   } else {
14     // updates
15     vm.$el = vm.__patch__(prevVnode, vnode)
16   }
17   activeInstance = prevActiveInstance
18   // update __vue__ reference
19   if (prevEl) {
20     prevEl.__vue__ = null
21   }
22   if (vm.$el) {
23     vm.$el.__vue__ = vm
24   }
25   // if parent is an HOC, update its $el as well
26   if (vm.$vnode && vm.$parent && vm.$vnode === vm.$parent._vnode) {
27     vm.$parent.$el = vm.$el
```

```

28   }
29   // updated hook is called by the scheduler to ensure that children are
30   // updated in a parent's updated hook.
31 }

```

`_update` 的核心就是调用 `vm.__patch__` 方法，这个方法实际上在不同的平台，比如 web 和 weex 上的定义是不一样的，因此在 web 平台中它的定义在

`src/platforms/web/runtime/index.js` 中：

```

1  Vue.prototype.__patch__ = inBrowser ? patch : noop

```

在 web 平台上，是否是服务端渲染也会对这个方法产生影响。因为在服务端渲染中，没有真实的浏览器 DOM 环境，所以不需要把 VNode 最终转换成 DOM，因此是一个空函数，而在浏览器端渲染中，它指向了 `patch` 方法，它的定义在 `src/platforms/web/runtime/patch.js` 中：

```

1  import * as nodeOps from 'web/runtime/node-ops'
2  import { createPatchFunction } from 'core/vdom/patch'
3  import baseModules from 'core/vdom/modules/index'
4  import platformModules from 'web/runtime/modules/index'
5
6  // the directive module should be applied last, after all
7  // built-in modules have been applied.
8  const modules = platformModules.concat(baseModules)
9
10 export const patch: Function = createPatchFunction({ nodeOps, modules })

```

该方法的定义是调用 `createPatchFunction` 方法的返回值，这里传入了一个对象，包含 `nodeOps` 参数和 `modules` 参数。其中，`nodeOps` 封装了一系列 DOM 操作的方法，`modules` 定义了一些模块的钩子函数的实现，我们这里先不详细介绍，来看一下 `createPatchFunction` 的实现，它定义在 `src/core/vdom/patch.js` 中：

```

1  const hooks = ['create', 'activate', 'update', 'remove', 'destroy']
2
3  export function createPatchFunction (backend) {
4    let i, j
5    const cbs = {}
6
7    const { modules, nodeOps } = backend
8
9    for (i = 0; i < hooks.length; ++i) {

```

```

10     cbs[hooks[i]] = []
11     for (j = 0; j < modules.length; ++j) {
12         if (isDef(modules[j][hooks[i]])) {
13             cbs[hooks[i]].push(modules[j][hooks[i]])
14         }
15     }
16 }
17
18 // ...
19
20 return function patch (oldVnode, vnode, hydrating, removeOnly) {
21     if (isUndef(vnode)) {
22         if (isDef(oldVnode)) invokeDestroyHook(oldVnode)
23         return
24     }
25
26     let isInitialPatch = false
27     const insertedVnodeQueue = []
28
29     if (isUndef(oldVnode)) {
30         // empty mount (likely as component), create new root element
31         isInitialPatch = true
32         createElm(vnode, insertedVnodeQueue)
33     } else {
34         const isRealElement = isDef(oldVnode.nodeType)
35         if (!isRealElement && sameVnode(oldVnode, vnode)) {
36             // patch existing root node
37             patchVnode(oldVnode, vnode, insertedVnodeQueue, removeOnly)
38         } else {
39             if (isRealElement) {
40                 // mounting to a real element
41                 // check if this is server-rendered content and if we can perform
42                 // a successful hydration.
43                 if (oldVnode.nodeType === 1 && oldVnode.hasAttribute(SSR_ATTR)) {
44                     oldVnode.removeAttribute(SSR_ATTR)
45                     hydrating = true
46                 }
47                 if (isTrue(hydrating)) {
48                     if (hydrate(oldVnode, vnode, insertedVnodeQueue)) {
49                         invokeInsertHook(vnode, insertedVnodeQueue, true)
50                         return oldVnode
51                     } else if (process.env.NODE_ENV !== 'production') {
52                         warn(
53                             'The client-side rendered virtual DOM tree is not matching ' +
54                             'server-rendered content. This is likely caused by incorrect '

```

+

```

55         'HTML markup, for example nesting block-level elements inside
    ' +
56         '<p>, or missing <tbody>. Bailing hydration and performing ' +
57         'full client-side render.'
58     )
59 }
60 }
61 // either not server-rendered, or hydration failed.
62 // create an empty node and replace it
63 oldVnode = emptyNodeAt(oldVnode)
64 }
65
66 // replacing existing element
67 const oldElm = oldVnode.elm
68 const parentElm = nodeOps.parentNode(oldElm)
69
70 // create new node
71 createElm(
72     vnode,
73     insertedVnodeQueue,
74     // extremely rare edge case: do not insert if old element is in a
75     // leaving transition. Only happens when combining transition +
76     // keep-alive + HOCs. (#4590)
77     oldElm._leaveCb ? null : parentElm,
78     nodeOps.nextSibling(oldElm)
79 )
80
81 // update parent placeholder node element, recursively
82 if (isDef(vnode.parent)) {
83     let ancestor = vnode.parent
84     const patchable = isPatchable(vnode)
85     while (ancestor) {
86         for (let i = 0; i < cbs.destroy.length; ++i) {
87             cbs.destroy[i](ancestor)
88         }
89         ancestor.elm = vnode.elm
90         if (patchable) {
91             for (let i = 0; i < cbs.create.length; ++i) {
92                 cbs.create[i](emptyNode, ancestor)
93             }
94             // #6513
95             // invoke insert hooks that may have been merged by create
96             hooks.
97             // e.g. for directives that uses the "inserted" hook.
98             const insert = ancestor.data.hook.insert
99             if (insert.merged) {

```

```

99          // start at index 1 to avoid re-invoking component mounted
          hook
100          for (let i = 1; i < insert.fns.length; i++) {
101              insert.fns[i]()
102          }
103      }
104      } else {
105          registerRef(ancestor)
106      }
107      ancestor = ancestor.parent
108  }
109  }
110
111      // destroy old node
112      if (isDef(parentElm)) {
113          removeVnodes(parentElm, [oldVnode], 0, 0)
114      } else if (isDef(oldVnode.tag)) {
115          invokeDestroyHook(oldVnode)
116      }
117  }
118  }
119
120      invokeInsertHook(vnode, insertedVnodeQueue, isInitialPatch)
121      return vnode.elm
122  }
123  }

```

createPatchFunction 内部定义了一系列的辅助方法，最终返回了一个 patch 方法，这个方法就赋值给了 vm._update 函数里调用的 `vm.__patch__`。

Q：为何 Vue.js 源码绕了这么一大圈，把相关代码分散到各个目录？

A：

1. 因为 patch 是平台相关的，在 Web 和 Weex 环境，它们把虚拟 DOM 映射到“平台 DOM”的方法是不同的，并且对“DOM”包括的属性模块创建和更新也不尽相同。因此每个平台都有各自的 `nodeOps` 和 `modules`，它们的代码需要托管在 src/platforms 这个大目录下；
2. 不同平台的 patch 的主要逻辑部分是相同的，所以这部分公共的部分托管在 core 这个大目录下。差异化部分只需要通过参数 `nodeOps` 和 `modules` 来区分：
 - a. nodeOps 表示对“平台 DOM”的一些操作方法；
 - b. modules 表示平台的一些模块，它们会在整个 patch 过程的不同阶段执行相应的钩子函数；

回到 patch 方法本身，它接收 4 个参数：

1. oldVnode 表示旧的 VNode 节点，它也可以不存在或者是一个 DOM 对象；
2. vnode 表示执行 _render 后返回的 VNode 的节点；
3. hydrating 表示是否是服务端渲染；
4. removeOnly 是给 transition-group 用的；

先来回顾我们的例子：

```
1  var app = new Vue({
2    el: '#app',
3    render: function (createElement) {
4      return createElement('div', {
5        attrs: {
6          id: 'app'
7        },
8      }, this.message)
9    },
10   data: {
11     message: 'Hello Vue!'
12   }
13 })
```

在 vm._update 的方法里是这么调用 patch 方法的：

```
1  // initial render
2  vm.$el = vm.__patch__(vm.$el, vnode, hydrating, false /* removeOnly */)
```

结合我们的例子：

1. 我们的场景是首次渲染，所以在执行 patch 函数的时候，传入的 vm.\$el 对应的是例子中 id 为 app 的 DOM 对象，这个也就是我们在 index.html 模板中写的 `<div id="app">`，vm.\$el 的赋值是在之前 mountComponent 函数做的；
2. vnode 对应的是调用 render 函数的返回值；
3. hydrating 在非服务端渲染情况下为 false；
4. removeOnly 为 false；

这时候回顾patch的执行过程上

```

1  const isRealElement = isDef(oldVnode.nodeType)
2  if (!isRealElement && sameVnode(oldVnode, vnode)) {
3    // patch existing root node
4    patchVnode(oldVnode, vnode, insertedVnodeQueue, removeOnly)
5  } else {
6    if (isRealElement) {
7      // mounting to a real element
8      // check if this is server-rendered content and if we can perform
9      // a successful hydration.
10     if (oldVnode.nodeType === 1 && oldVnode.hasAttribute(SSR_ATTR)) {
11       oldVnode.removeAttribute(SSR_ATTR)
12       hydrating = true
13     }
14     if (isTrue(hydrating)) {
15       if (hydrate(oldVnode, vnode, insertedVnodeQueue)) {
16         invokeInsertHook(vnode, insertedVnodeQueue, true)
17         return oldVnode
18       } else if (process.env.NODE_ENV !== 'production') {
19         warn(
20           'The client-side rendered virtual DOM tree is not matching ' +
21           'server-rendered content. This is likely caused by incorrect ' +
22           'HTML markup, for example nesting block-level elements inside ' +
23           '<p>, or missing <tbody>. Bailing hydration and performing ' +
24           'full client-side render.'
25         )
26       }
27     }
28     // either not server-rendered, or hydration failed.
29     // create an empty node and replace it
30     oldVnode = emptyNodeAt(oldVnode)
31   }
32
33   // replacing existing element
34   const oldElm = oldVnode.elm
35   const parentElm = nodeOps.parentNode(oldElm)
36
37   // create new node
38   createElm(
39     vnode,
40     insertedVnodeQueue,
41     // extremely rare edge case: do not insert if old element is in a
42     // leaving transition. Only happens when combining transition +
43     // keep-alive + HOCs. (#4590)
44     oldElm._leaveCb ? null : parentElm,
45     nodeOps.nextSibling(oldElm)
46   )

```

由于我们传入的 `oldVnode` 实际上是一个 DOM，所以 `isRealElement` 为 `true`，接下来又通过 `emptyNodeAt` 方法把 `oldVnode` 转换成 `VNode` 对象，然后再调用 `createElm` 方法，来看一下它的实现：

```

1  function createElm (
2    vnode,
3    insertedVnodeQueue,
4    parentElm,
5    refElm,
6    nested,
7    ownerArray,
8    index
9  ) {
10   if (isDef(vnode.elm) && isDef(ownerArray)) {
11     // This vnode was used in a previous render!
12     // now it's used as a new node, overwriting its elm would cause
13     // potential patch errors down the road when it's used as an insertion
14     // reference node. Instead, we clone the node on-demand before creating
15     // associated DOM element for it.
16     vnode = ownerArray[index] = cloneVNode(vnode)
17   }
18
19   vnode.isRootInsert = !nested // for transition enter check
20   if (createComponent(vnode, insertedVnodeQueue, parentElm, refElm)) {
21     return
22   }
23
24   const data = vnode.data
25   const children = vnode.children
26   const tag = vnode.tag
27   if (isDef(tag)) {
28     if (process.env.NODE_ENV !== 'production') {
29       if (data && data.pre) {
30         creatingElmInVPre++
31       }
32       if (isUnknownElement(vnode, creatingElmInVPre)) {
33         warn(
34           'Unknown custom element: <' + tag + '> - did you ' +
35           'register the component correctly? For recursive components, ' +
36           'make sure to provide the "name" option.',
37           vnode.context
38         )
39       }

```

```

40     }
41
42     vnode.elm = vnode.ns
43       ? nodeOps.createElementNS(vnode.ns, tag)
44       : nodeOps.createElement(tag, vnode)
45     setScope(vnode)
46
47     /* istanbul ignore if */
48     if (___WEEX___) {
49       // ...
50     } else {
51       createChildren(vnode, children, insertedVnodeQueue)
52       if (isDef(data)) {
53         invokeCreateHooks(vnode, insertedVnodeQueue)
54       }
55       insert(parentElm, vnode.elm, refElm)
56     }
57
58     if (process.env.NODE_ENV !== 'production' && data && data.pre) {
59       creatingElmInVPre--
60     }
61   } else if (isTrue(vnode.isComment)) {
62     vnode.elm = nodeOps.createComment(vnode.text)
63     insert(parentElm, vnode.elm, refElm)
64   } else {
65     vnode.elm = nodeOps.createTextNode(vnode.text)
66     insert(parentElm, vnode.elm, refElm)
67   }
68 }

```

createElm 的作用是通过虚拟节点创建真实的 DOM 并插入到它的父节点中。

1. createComponent 方法目的是尝试创建子组件，后面详细解释，在当前这个 case 下它的返回值为 false；
2. 接下来判断 vnode 是否包含 tag，如果包含，先简单对 tag 的合法性在非生产环境下做校验，看是否是一个合法标签；然后再去调用平台 DOM 的操作去创建一个占位符元素；（ns: nameSpace）

```

1  vnode.elm = vnode.ns
2    ? nodeOps.createElementNS(vnode.ns, tag)
3    : nodeOps.createElement(tag, vnode)

```

接下来调用 createChildren 方法去创建子元素，也就是去遍历 createElm 的：

```

1  createChildren(vnode, children, insertedVnodeQueue)
2
3  function createChildren (vnode, children, insertedVnodeQueue) {
4    if (Array.isArray(children)) {
5      if (process.env.NODE_ENV !== 'production') {
6        checkDuplicateKeys(children)
7      }
8      for (let i = 0; i < children.length; ++i) {
9        createElm(children[i], insertedVnodeQueue, vnode.elm, null, true,
children, i)
10      }
11    } else if (isPrimitive(vnode.text)) {
12      nodeOps.appendChild(vnode.elm, nodeOps.createTextNode(String(vnode.text)))
13    }
14  }

```

接着再调用 `invokeCreateHooks` 方法执行所有的 `create` 的钩子并把 `vnode` push 到 `insertedVnodeQueue` 中。

```

1  if (isDef(data)) {
2    invokeCreateHooks(vnode, insertedVnodeQueue)
3  }
4
5  function invokeCreateHooks (vnode, insertedVnodeQueue) {
6    for (let i = 0; i < cbs.create.length; ++i) {
7      cbs.create[i](emptyNode, vnode)
8    }
9    i = vnode.data.hook // Reuse variable
10   if (isDef(i)) {
11     if (isDef(i.create)) i.create(emptyNode, vnode)
12     if (isDef(i.insert)) insertedVnodeQueue.push(vnode)
13   }
14 }

```

最后调用 `insert` 方法把 DOM 插入到父节点中，因为是递归调用，子元素会优先调用 `insert`，所以整个 `vnode` 树节点的插入顺序是先子后父。来看一下 `insert` 方法，它的定义在 `src/core/vdom/patch.js` 上。

```

1  insert(parentElm, vnode.elm, refElm)
2
3  function insert (parent, elm, ref) {
4    if (isDef(parent)) {
5      if (isDef(ref)) {

```

```

6      if (ref.parentNode === parent) {
7          nodeOps.insertBefore(parent, elm, ref)
8      }
9      } else {
10         nodeOps.appendChild(parent, elm)
11     }
12 }
13 }

```

insert 调用一些 nodeOps 把子节点插入到父节点中，这些辅助方法定义在 `src/platforms/web/runtime/node-ops.js` 中：

```

1  export function insertBefore (parentNode: Node, newNode: Node, referenceNode:
    Node) {
2      parentNode.insertBefore(newNode, referenceNode)
3  }
4
5  export function appendChild (node: Node, child: Node) {
6      node.appendChild(child)
7  }

```

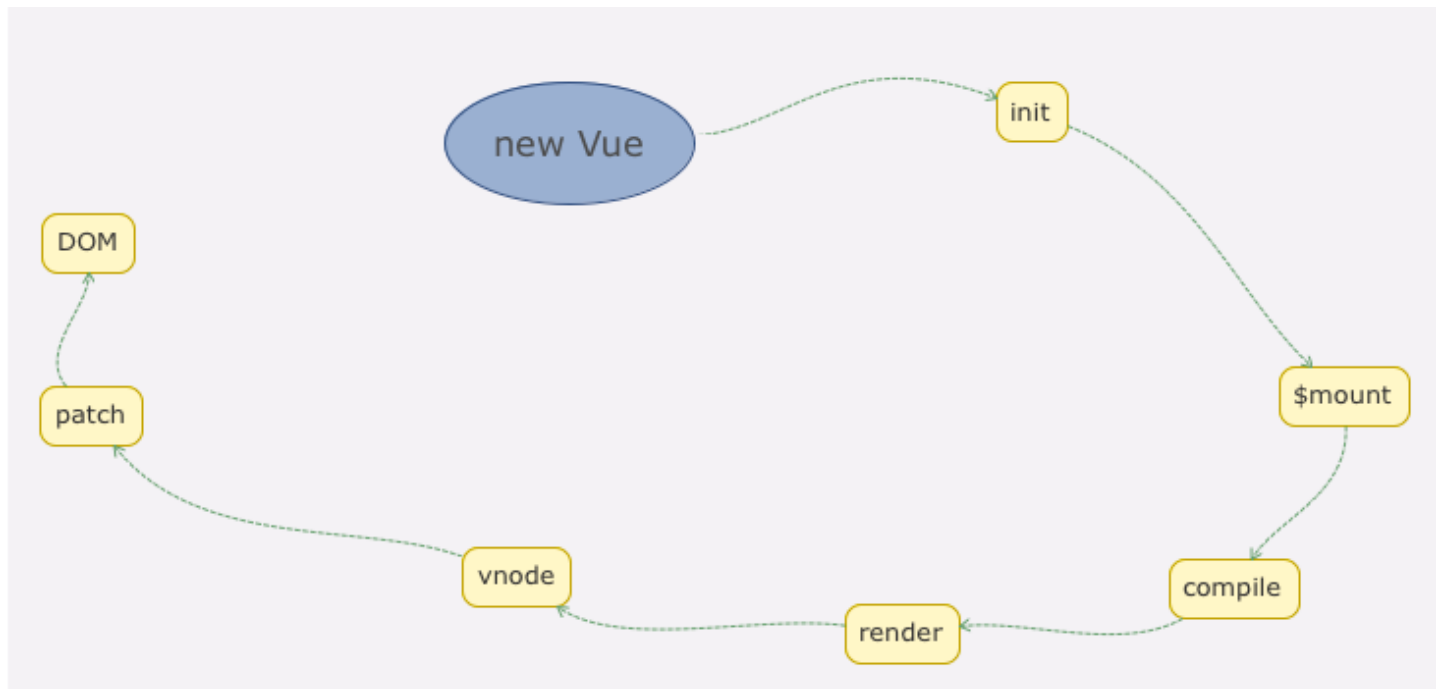
其实在web中，就是调用原生 DOM 的 API 进行 DOM 操作；

在 `createElm` 过程中，如果 vnode 节点不包含 tag，则它有可能是一个注释或者纯文本节点，可以直接插入到父元素中。在我们这个例子中，最内层就是一个文本 vnode，它的 text 值取的就是之前的 this.message 的值 Hello Vue!。

再回到 `patch` 方法，首次渲染我们调用了 createElm 方法，这里传入的 parentElm 是 oldVnode.elm 的父元素，在我们的例子是 id 为 #app div 的父元素，也就是 Body；实际上整个过程就是递归创建了一个完整的 DOM 树并插入到 Body 上；

最后，我们根据之前递归 createElm 生成的 vnode 插入顺序队列，执行相关的 insert 钩子函数；

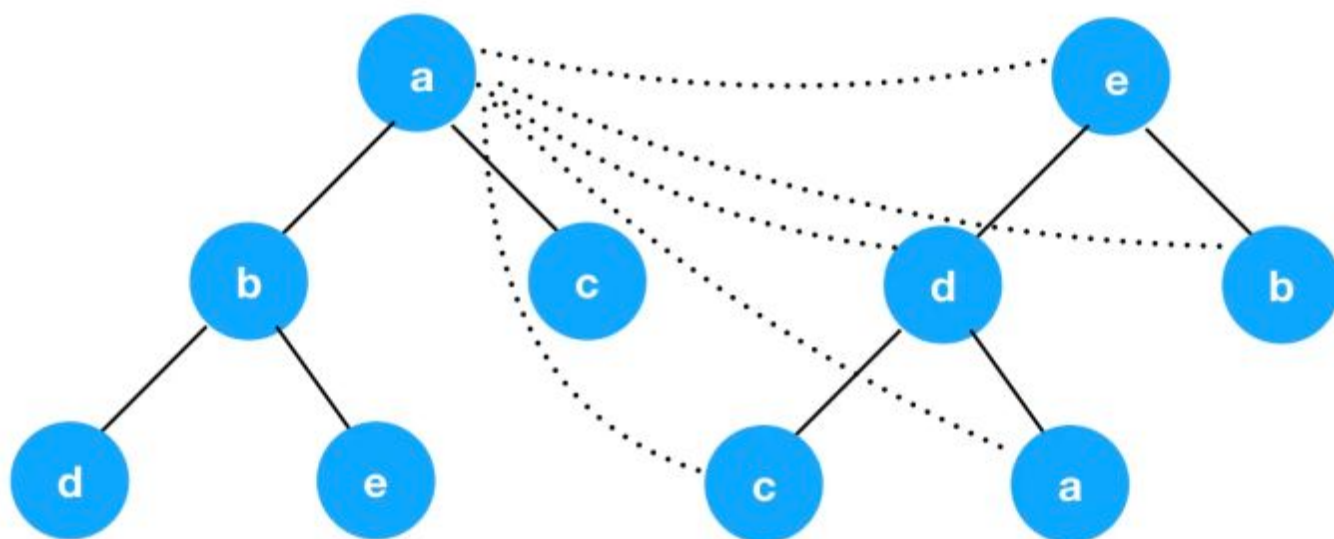
以上就是数据在Vue2中如何渲染成DOM的过程。



Vue2 Diff 算法

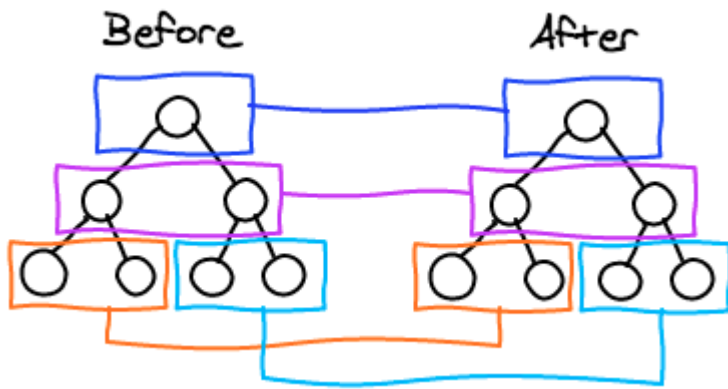
Diff 算法的目的

diff算法的目的是为了找到哪些节点发生了变化，哪些节点没有发生变化可以复用。如果用最传统的diff算法，如下图所示，每个节点都要遍历另一棵树上的所有节点做比较，这就是 $O(n^2)$ 的复杂度，加上更新节点时的 $O(n)$ 复杂度，那就总共达到了 $O(n^3)$ 的复杂度，这对于一个结构复杂节点数众多的页面，成本是非常大的。



实际上vue和react都对虚拟dom的diff算法做了一定的优化，将复杂度降低到了 $O(n)$ 级别，具体的策略是：同层的节点才相互比较；

1. 节点比较时，如果类型不同，则对该节点及其所有子节点直接销毁新建；
2. 类型相同的子节点，使用key帮助查找，并且使用算法优化查找效率。其中react和vue2以及vue3的diff算法都不尽相同；



主要对比Vue2和Vue3，掌握为什么要从Vue2升级到Vue3，并代入后续代码，掌握Vue实现diff的流程；

前提：

- `mount(vnode, parent, [refNode])` : 通过 `vnode` 生成真实的DOM节点。parent为其父级的真实DOM节点，`refNode` 为真实的DOM节点，其父级节点为parent。如果refNode不为空，vnode生成的DOM节点就会插入到refNode之前；如果refNode为空，那么vnode生成的DOM节点就作为最后一个子节点插入到parent中
- `patch(prevNode, nextNode, parent)` : 可以简单的理解为给当前DOM节点进行更新，并且调用diff算法对比自身的子节点；

双端比较

双端比较就是新列表和旧列表两个列表的头与尾互相对比，，在对比的过程中指针会逐渐向内靠拢，直到某一个列表的节点全部遍历过，对比停止；

patch

先判断是否是首次渲染，如果是首次渲染那么我们就直接createElm即可；如果不是就去判断新老两个节点的元素类型否一样；如果两个节点都是一样的，那么就深入检查他们的子节点。如果两个节点不一样那就说明Vnode完全被改变了，就可以直接替换oldVnode；

先判断是否是首次渲染，如果是首次渲染那么我们就直接createElm即可；如果不是就去判断新老两个节点的元素类型否一样；如果两个节点都是一样的，那么就深入检查他们的子节点。如果两个节点不一样那就说明Vnode完全被改变了，就可以直接替换oldVnode；

```

1  function patch(oldVnode, vnode, hydrating, removeOnly) {
2      // 判断新的vnode是否为空
3      if (isUndef(vnode)) {
4          // 如果老的vnode不为空 卸载所有的老vnode
5          if (isDef(oldVnode)) invokeDestroyHook(oldVnode)
6          return
7      }
8      let isInitialPatch = false
9      // 用来存储 insert钩子函数，在插入节点之前调用
10     const insertedVnodeQueue = []
11     // 如果老节点不存在，直接创建新节点
12     if (isUndef(oldVnode)) {
13         isInitialPatch = true
14         createElm(vnode, insertedVnodeQueue)
15     } else {
16         // 是不是元素节点
17         const isRealElement = isDef(oldVnode.nodeType)
18         // 当老节点不是真实的DOM节点，并且新老节点的type和key相同，进行patchVnode更新工
作
19         if (!isRealElement && sameVnode(oldVnode, vnode)) {
20             patchVnode(oldVnode, vnode, insertedVnodeQueue, null, null,
removeOnly)
21         } else {
22             // 如果不是同一元素节点的话
23             // 当老节点是真实DOM节点的时候
24             if (isRealElement) {
25                 // 如果是元素节点 并且在SSR环境的时候 修改SSR_ATTR属性
26                 if (oldVnode.nodeType === 1 && oldVnode.hasAttribute(SSR_ATTR)) {
27                     // 就是服务端渲染的，删掉这个属性
28                     oldVnode.removeAttribute(SSR_ATTR)
29                     hydrating = true
30                 }
31                 // 这个判断里是服务端渲染的处理逻辑
32                 if (isTrue(hydrating)) {
33                     if (hydrate(oldVnode, vnode, insertedVnodeQueue)) {
34                         invokeInsertHook(vnode, insertedVnodeQueue, true)
35                         return oldVnode
36                     }
37                 }
38                 // 如果不是服务端渲染的，或者混合失败，就创建一个空的注释节点替换 oldVnode
39                 oldVnode = emptyNodeAt(oldVnode)
40             }
41
42             // 拿到 oldVnode 的父节点
43             const oldElm = oldVnode.elm
44             const parentElm = nodeOps.parentNode(oldElm)
45

```

```

46 // 根据新的 vnode 创建一个 DOM 节点, 挂载到父节点上
47 createElm(
48     vnode,
49     insertedVnodeQueue,
50     oldElm._leaveCb ? null : parentElm,
51     nodeOps.nextSibling(oldElm)
52 )
53 // 如果新的 vnode 的根节点存在, 就是说根节点被修改了, 就需要遍历更新父节点
54 // 递归 更新父占位符元素
55 // 就是执行一遍 父节点的 destroy 和 create、insert 的 钩子函数
56 if (isDef(vnode.parent)) {
57     let ancestor = vnode.parent
58     const patchable = isPatchable(vnode)
59     // 更新父组件的占位元素
60     while (ancestor) {
61         // 卸载老根节点下的全部组件
62         for (let i = 0; i < cbs.destroy.length; ++i) {
63             cbs.destroy[i](ancestor)
64         }
65         // 替换现有元素
66         ancestor.elm = vnode.elm
67         if (patchable) {
68             for (let i = 0; i < cbs.create.length; ++i) {
69                 cbs.create[i](emptyNode, ancestor)
70             }
71             // #6513
72             // invoke insert hooks that may have been merged by create
73             hooks.
74                 // e.g. for directives that uses the "inserted" hook.
75                 const insert = ancestor.data.hook.insert
76                 if (insert.merged) {
77                     // start at index 1 to avoid re-invoking component mounted
78                     hook
79                     for (let i = 1; i < insert.fns.length; i++) {
80                         insert.fns[i]()
81                     }
82                 } else {
83                     registerRef(ancestor)
84                 }
85             // 更新父节点
86             ancestor = ancestor.parent
87         }
88     }
89     // 如果旧节点还存在, 就删掉旧节点
90     if (isDef(parentElm)) {
91         removeVnodes([oldVnode], 0, 0)

```

```

91         } else if (isDef(oldVnode.tag)) {
92             // 否则直接卸载 oldVnode
93             invokeDestroyHook(oldVnode)
94         }
95     }
96 }
97 // 执行 虚拟 dom 的 insert 钩子函数
98 invokeInsertHook(vnode, insertedVnodeQueue, isInitialPatch)
99 // 返回最新 vnode 的 elm , 也就是真实的 dom节点
100 return vnode.elm
101 }

```

patchVnode

- 如果 `Vnode` 和 `oldVnode` 指向同一个对象，则直接return即可；
- 将旧节点的真实 DOM 赋值到新节点（真实 dom 连线到新子节点）称为elm，然后遍历调用 update 更新 `oldVnode` 上的所有属性，比如 class,style,attrs,domProps,events...；
- 如果新老节点都有文本节点，并且文本不相同，那么就用 `vnode` .text更新文本内容。
- 如果oldVnode有子节点而 `Vnode` 没有，则直接删除老节点即可；
- 如果oldVnode没有子节点而 `Vnode` 有，则将Vnode的子节点真实化之后添加到DOM中即可。
- 如果两者都有子节点，则执行 `updateChildren` 函数比较子节点。

```

1  function patchVnode(
2      oldVnode, // 老的虚拟 DOM 节点
3      vnode, // 新节点
4      insertedVnodeQueue, // 插入节点队列
5      ownerArray, // 节点数组
6      index, // 当前节点的下标
7      removeOnly
8  ) {
9      // 新老节点对比地址一样，直接跳过
10     if (oldVnode === vnode) {
11         return
12     }
13     if (isDef(vnode.elm) && isDef(ownerArray)) {
14         // clone reused vnode
15         vnode = ownerArray[index] = cloneVNode(vnode)
16     }
17     const elm = vnode.elm = oldVnode.elm
18     // 如果当前节点是注释或 v-if 的，或者是异步函数，就跳过检查异步组件
19     if (isTrue(oldVnode.isAsyncPlaceholder)) {

```

```

20     if (isDef(vnode.asyncFactory.resolved)) {
21       hydrate(oldVnode.elm, vnode, insertedVnodeQueue)
22     } else {
23       vnode.isAsyncPlaceholder = true
24     }
25     return
26   }
27   // 当前节点是静态节点的时候, key 也一样, 或者有 v-once 的时候, 就直接赋值返回
28   if (isTrue(vnode.isStatic) &&
29     isTrue(oldVnode.isStatic) &&
30     vnode.key === oldVnode.key &&
31     (isTrue(vnode.isCloned) || isTrue(vnode.isOnce))) {
32   } {
33     vnode.componentInstance = oldVnode.componentInstance
34     return
35   }
36   let i
37   const data = vnode.data
38   if (isDef(data) && isDef(i = data.hook) && isDef(i = i.prepatch)) {
39     i(oldVnode, vnode)
40   }
41   const oldCh = oldVnode.children
42   const ch = vnode.children
43   if (isDef(data) && isPatchable(vnode)) {
44     // 遍历调用 update 更新 oldVnode 所有属性, 比如
45     // class, style, attrs, domProps, events...
46     // 这里的 update 钩子函数是 vnode 本身的钩子函数
47     for (i = 0; i < cbs.update.length; ++i) cbs.update[i](oldVnode, vnode)
48     // 这里的 update 钩子函数是我们传过来的函数
49     if (isDef(i = data.hook) && isDef(i = i.update)) i(oldVnode, vnode)
50   }
51   // 如果新节点不是文本节点, 也就是说有子节点
52   if (isUndef(vnode.text)) {
53     // 如果新老节点都有子节点
54     if (isDef(oldCh) && isDef(ch)) {
55       // 如果新老节点的子节点不一样, 就执行 updateChildren 函数, 对比子节点
56       if (oldCh !== ch) updateChildren(elm, oldCh, ch, insertedVnodeQueue,
57         removeOnly)
58     } else if (isDef(ch)) {
59       // 如果新节点有子节点的话, 就是说老节点没有子节点
60       // 如果老节点是文本节点, 就是说没有子节点, 就清空
61       if (isDef(oldVnode.text)) nodeOps.setTextContent(elm, '')
62       // 添加新节点
63       addVnodes(elm, null, ch, 0, ch.length - 1, insertedVnodeQueue)
64     } else if (isDef(oldCh)) {
65       // 如果新节点没有子节点, 老节点有子节点, 就删除

```

```

65         removeVnodes(oldCh, 0, oldCh.length - 1)
66     } else if (isDef(oldVnode.text)) {
67         // 如果老节点是文本节点，就清空
68         nodeOps.setTextContent(elm, '')
69     }
70 } else if (oldVnode.text !== vnode.text) {
71     // 如果老节点的文本和新节点的文本不同，就更新文本
72     nodeOps.setTextContent(elm, vnode.text)
73 }
74 if (isDef(data)) {
75     if (isDef(i = data.hook) && isDef(i = i.postpatch)) i(oldVnode, vnode)
76 }
77 }

```

updateChildren

为了方便理解，这里手动实现Vue2中的updateChildren

实现思路

我们先用四个指针指向两个列表的头尾

```

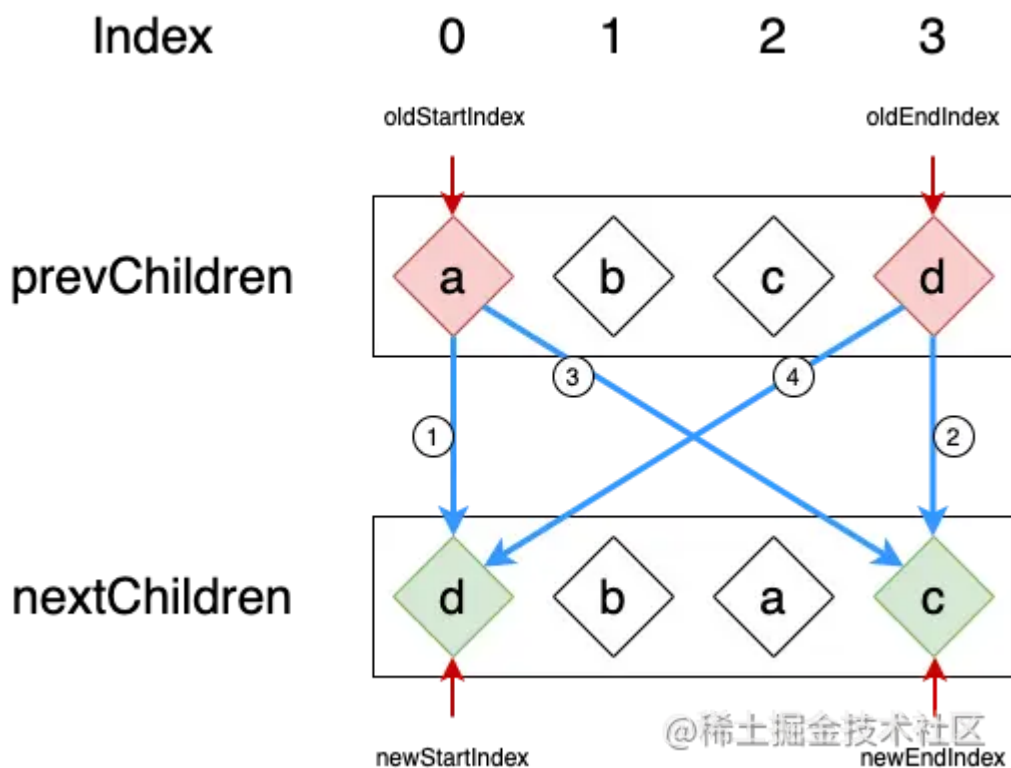
1  function vue2Diff(prevChildren, nextChildren, parent) {
2      let oldStartIndex = 0,
3          oldEndIndex = prevChildren.length - 1,
4          newStartIndex = 0,
5          newEndIndex = nextChildren.length - 1;
6      let oldStartNode = prevChildren[oldStartIndex],
7          oldEndNode = prevChildren[oldEndIndex],
8          newStartNode = nextChildren[nextStartIndex],
9          newEndNode = nextChildren[nextEndIndex];
10 }

```

根据四个指针找到四个节点，然后进行对比，那么如何对比呢？我们按照以下四个步骤进行对比

1. 使用旧列表的头一个节点 `oldStartNode` 与新列表的头一个节点 `newStartNode` 对比；
2. 使用旧列表的最后一个节点 `oldEndNode` 与新列表的最后一个节点 `newEndNode` 对比；
3. 使用旧列表的头一个节点 `oldStartNode` 与新列表的最后一个节点 `newEndNode` 对比；
4. 使用旧列表的最后一个节点 `oldEndNode` 与新列表的头一个节点 `newStartNode` 对比；

使用以上四步进行对比，去寻找key相同的可复用的节点，当在某一步中找到了则停止后面的寻找。具体对比顺序如下图：



对比顺序代码结构如下：

```
1  function vue2Diff(prevChildren, nextChildren, parent) {
2    let oldStartIndex = 0,
3        oldEndIndex = prevChildren.length - 1,
4        newStartIndex = 0,
5        newEndIndex = nextChildren.length - 1;
6    let oldStartNode = prevChildren[oldStartIndex],
7        oldEndNode = prevChildren[oldEndIndex],
8        newStartNode = nextChildren[newStartIndex],
9        newEndNode = nextChildren[newEndIndex];
10
11    if (oldStartNode.key === newStartNode.key) {
12
13    } else if (oldEndNode.key === newEndNode.key) {
14
15    } else if (oldStartNode.key === newEndNode.key) {
16
17    } else if (oldEndNode.key === newStartNode.key) {
18
19    }
20 }
```

当对比时找到了可复用的节点，我们还是先 `patch` 给元素打补丁，然后将指针进行前/后移一位指针。根据对比节点的不同，我们移动的指针和方向也不同，具体规则如下：

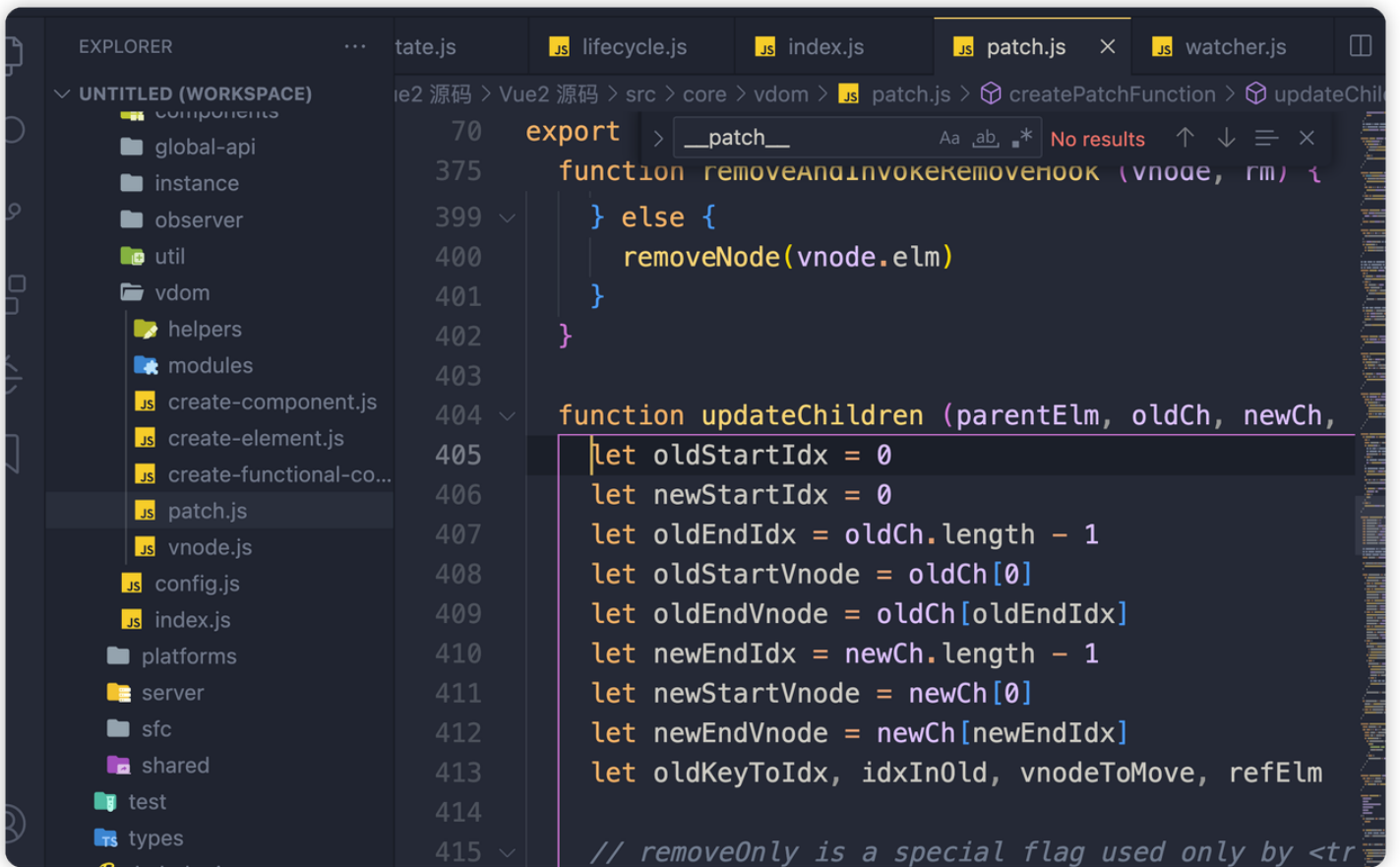
1. 当旧列表的头一个节点 `oldStartNode` 与新列表的头一个节点 `newStartNode` 对比时key相同。那么旧列表的头指针 `oldStartIndex` 与新列表的头指针 `newStartIndex` 同时向后移动一位；
2. 当旧列表的最后一个节点 `oldEndNode` 与新列表的最后一个节点 `newEndNode` 对比时key相同。那么旧列表的尾指针 `oldEndIndex` 与新列表的尾指针 `newEndIndex` 同时向前移动一位；
3. 当旧列表的头一个节点 `oldStartNode` 与新列表的最后一个节点 `newEndNode` 对比时key相同。那么旧列表的头指针 `oldStartIndex` 向后移动一位；新列表的尾指针 `newEndIndex` 向前移动一位；
4. 当旧列表的最后一个节点 `oldEndNode` 与新列表的头一个节点 `newStartNode` 对比时key相同。那么旧列表的尾指针 `oldEndIndex` 向前移动一位；新列表的头指针 `newStartIndex` 向后移动一位；

```
1  function vue2Diff(prevChildren, nextChildren, parent) {
2    let oldStartIndex = 0,
3        oldEndIndex = prevChildren.length - 1,
4        newStartIndex = 0,
5        newEndIndex = nextChildren.length - 1;
6    let oldStartNode = prevChildren[oldStartIndex],
7        oldEndNode = prevChildren[oldEndIndex],
8        newStartNode = nextChildren[newStartIndex],
9        newEndNode = nextChildren[newEndIndex];
10
11    if (oldStartNode.key === newStartNode.key) {
12      patch(oldStartNode, newStartNode, parent)
13
14      oldStartIndex++
15      newStartIndex++
16      oldStartNode = prevChildren[oldStartIndex]
17      newStartNode = nextChildren[newStartIndex]
18    } else if (oldEndNode.key === newEndNode.key) {
19      patch(oldEndNode, newEndNode, parent)
20
21      oldEndIndex--
22      newEndIndex--
23      oldEndNode = prevChildren[oldEndIndex]
24      newEndNode = nextChildren[newEndIndex]
25    } else if (oldStartNode.key === newEndNode.key) {
26      patch(oldStartNode, newEndNode, parent)
27
28      oldStartIndex++
29      newEndIndex--
```

```

30     oldStartNode = prevChildren[oldStartIndex]
31     newEndNode = nextChildren[newEndIndex]
32   } else if (oldEndNode.key === newStartNode.key) {
33     patch(oldEndNode, newStartNode, parent)
34
35     oldEndIndex--
36     nextStartIndex++
37     oldEndNode = prevChildren[oldEndIndex]
38     newStartNode = nextChildren[newStartIndex]
39   }
40 }

```



跟 React 先比较还是差了点意思，React 是先标记，最后 commitRoot

```
29 } from > __patch__ Aa ab_* No results ↑ ↓ ≡ ×
30
31 export const emptyNode = new VNode('', {}, [])
32
33 const hooks = ['create', 'activate', 'update', 'remo
34 ⚡
35 function sameVnode(a, b) { "Vnode": Unknown word
36   return (
37     a.key === b.key &&
38     a.asyncFactory === b.asyncFactory && (
39       (
40         a.tag === b.tag &&
41         a.isComment === b.isComment &&
42         isDef(a.data) === isDef(b.data) &&
43         sameInputType(a, b)
44       ) || (
45         isTrue(a.isAsyncPlaceholder) &&
46         isUndef(b.asyncFactory.error)
47       )
48     )
49   )
50 }
```

上面提到，要让指针向内靠拢，所以我们需要循环。循环停止的条件是当其中一个列表的节点全部遍历完成，代码如下：

```
1  function vue2Diff(prevChildren, nextChildren, parent) {
2    let oldStartIndex = 0,
3        oldEndIndex = prevChildren.length - 1,
4        newStartIndex = 0,
5        newEndIndex = nextChildren.length - 1;
6    let oldStartNode = prevChildren[oldStartIndex],
7        oldEndNode = prevChildren[oldEndIndex],
8        newStartNode = nextChildren[newStartIndex],
9        newEndNode = nextChildren[newEndIndex];
10   while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
11     if (oldStartNode.key === newStartNode.key) {
12       patch(oldStartNode, newStartNode, parent)
13     }
14     oldStartIndex++
15     newStartIndex++
16     oldStartNode = prevChildren[oldStartIndex]
17     newStartNode = nextChildren[newStartIndex]
18   } else if (oldEndNode.key === newEndNode.key) {
19     patch(oldEndNode, newEndNode, parent)
20   }
```

```

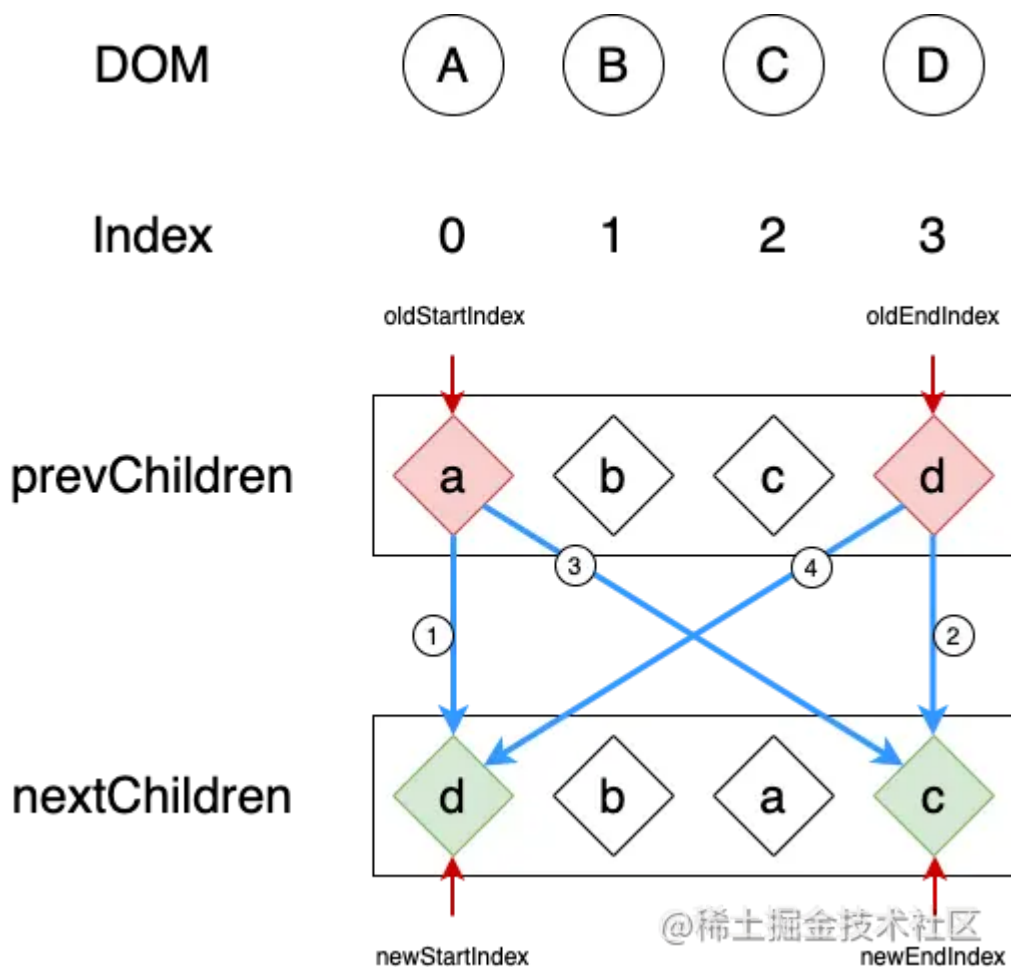
21     oldEndIndex--
22     newndIndex--
23     oldEndNode = prevChildren[oldEndIndex]
24     newEndNode = nextChildren[newEndIndex]
25   } else if (oldStartNode.key === newEndNode.key) {
26     patch(oldvStartNode, newEndNode, parent)
27
28     oldStartIndex++
29     newEndIndex--
30     oldStartNode = prevChildren[oldStartIndex]
31     newEndNode = nextChildren[newEndIndex]
32   } else if (oldEndNode.key === newStartNode.key) {
33     patch(oldEndNode, newStartNode, parent)
34
35     oldEndIndex--
36     newStartIndex++
37     oldEndNode = prevChildren[oldEndIndex]
38     newStartNode = nextChildren[newStartIndex]
39   }
40 }
41 }

```

至此整体的循环我们就全部完成了，下面我们需要考虑这样两个问题：

- 什么情况下DOM节点需要移动；
- DOM节点如何移动；

我们来解决第一个问题：什么情况下需要移动，我们还是以上图为例：



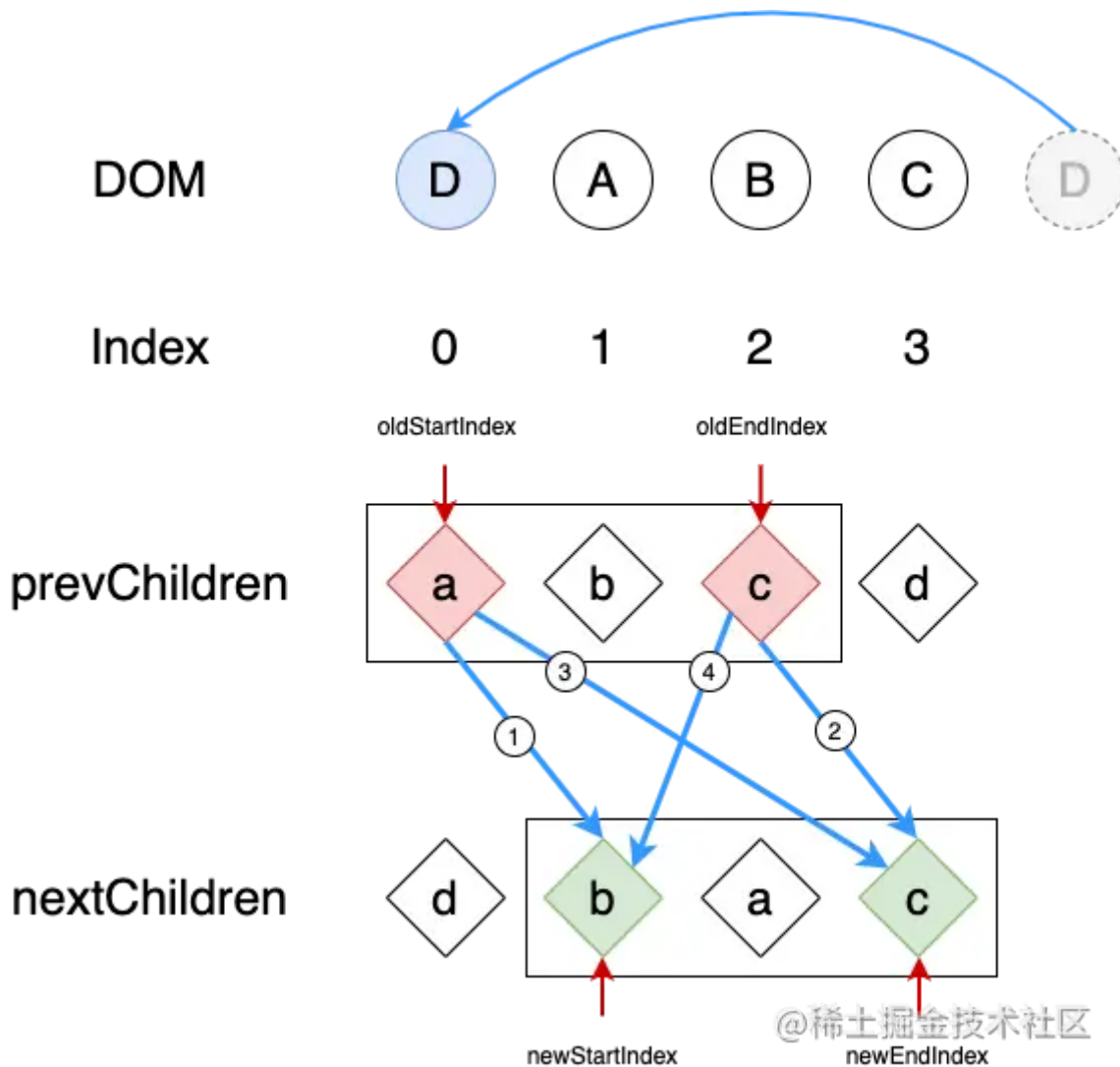
当我们在第一个循环时，在第四步发现旧列表的尾节点 `oldEndNode` 与新列表的头节点 `newStartNode` 的key相同，是可复用的DOM节点。通过观察我们可以发现，原本在旧列表末尾的节点，却是新列表中的开头节点，没有人比他更靠前，因为他是第一个，所以我们只需要把当前的节点移动到原本旧列表中的第一个节点之前，让它成为第一个节点即可。

```

1  function vue2Diff(prevChildren, nextChildren, parent) {
2    // ...
3    while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
4      if (oldStartNode.key === newStartNode.key) {
5        // ...
6      } else if (oldEndNode.key === newEndNode.key) {
7        // ...
8      } else if (oldStartNode.key === newEndNode.key) {
9        // ...
10     } else if (oldEndNode.key === newStartNode.key) {
11       patch(oldEndNode, newStartNode, parent)
12       // 移动到旧列表头节点之前
13       parent.insertBefore(oldEndNode.el, oldStartNode.el)
14
15       oldEndIndex--
16       newStartIndex++
17       oldEndNode = prevChildren[oldEndIndex]
18       newStartNode = nextChildren[newStartIndex]

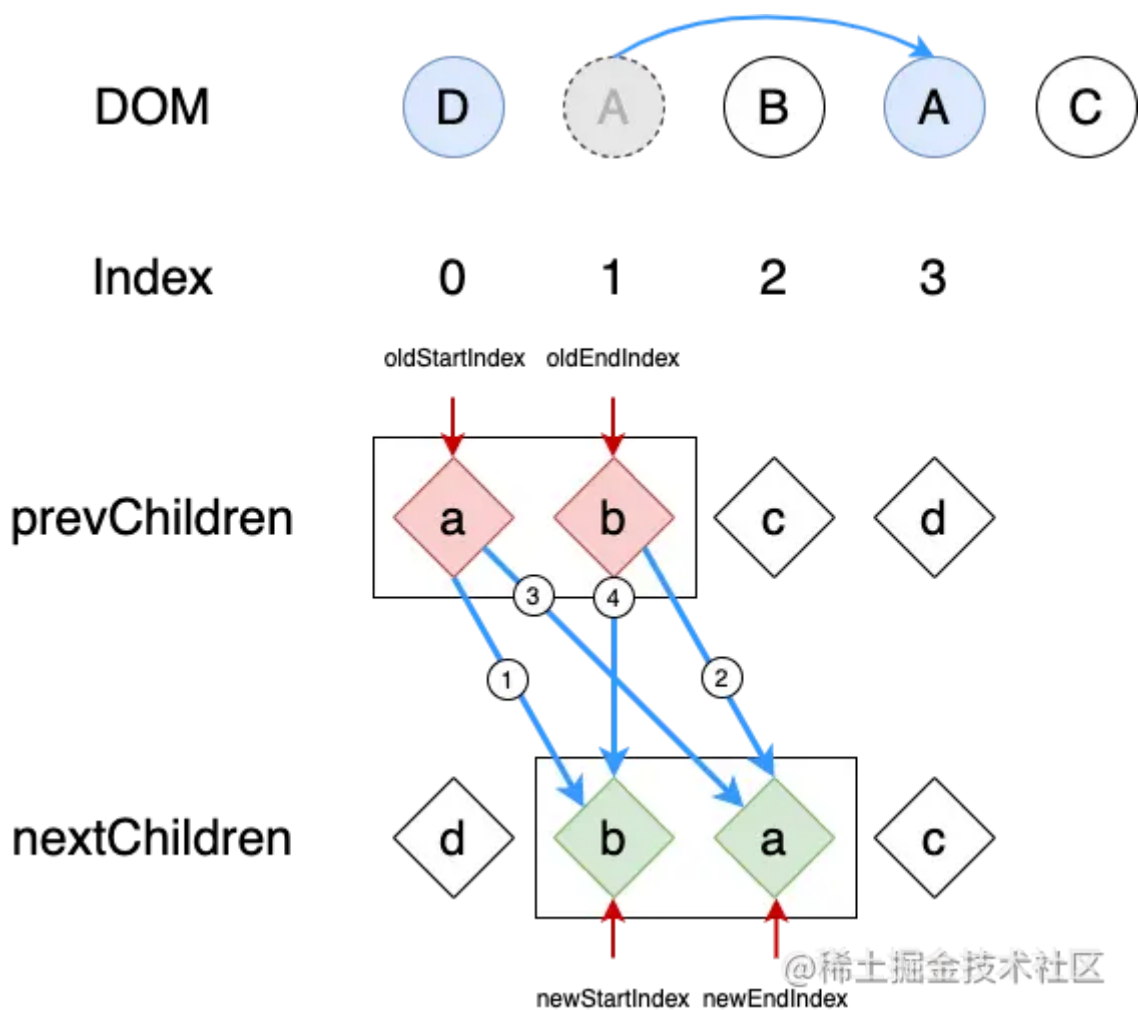
```

```
19     }  
20     }  
21 }
```



进入第二次循环，我们在第二步发现，旧列表的尾节点 `oldEndNode` 和新列表的尾节点 `newEndNode` 为复用节点。原本在旧列表中就是尾节点，在新列表中也是尾节点，说明该节点不需要移动，所以我们什么都不需要做。

同理，如果是旧列表的头节点 `oldStartNode` 和新列表的头节点 `newStartNode` 为复用节点，我们也什么都不需要做



进入第三次循环，我们在第三部发现，旧列表的头节点 `oldStartNode` 和新列表的尾节点 `newEndNode` 为复用节点。，我们只要将DOM-A移动到DOM-B后面就可以了。

依照惯例我们还是解释一下，原本旧列表中是头节点，然后在新列表中是尾节点。那么只要在旧列表中把当前的节点移动到原本尾节点的后面，就可以了。

```

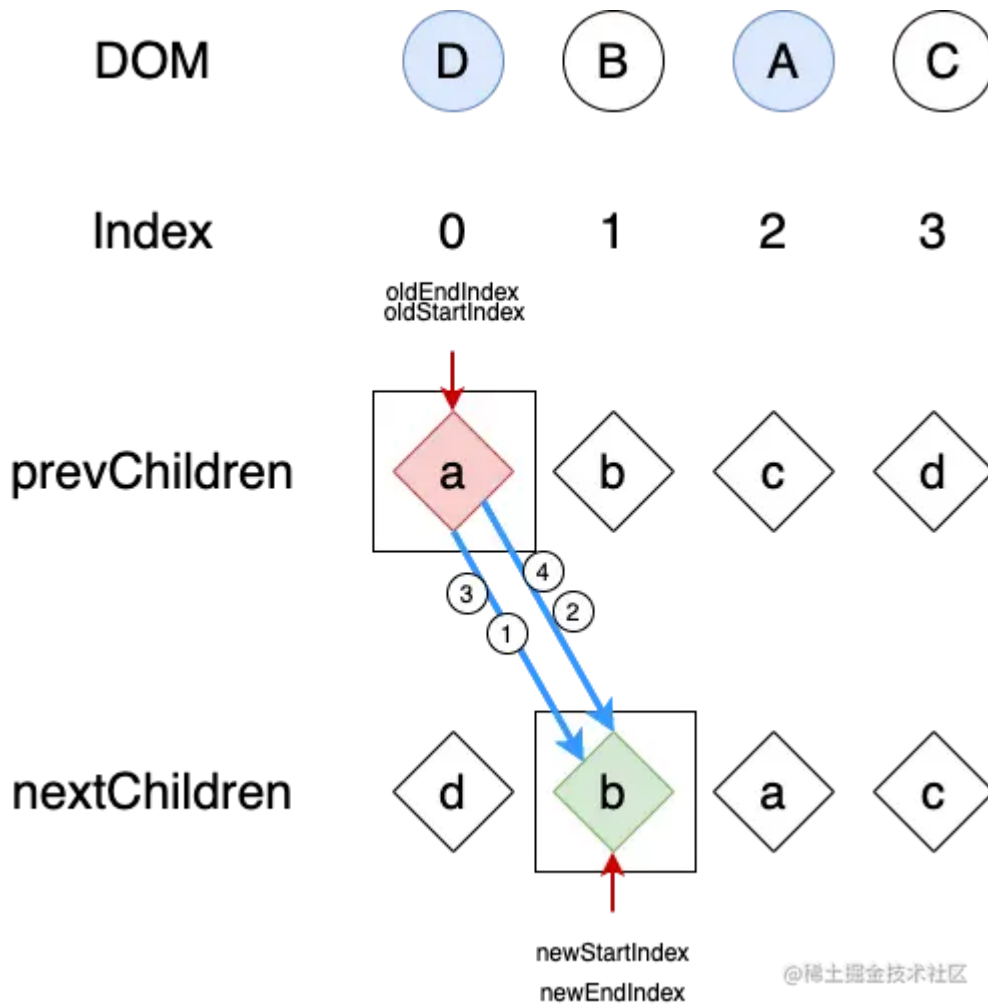
1  function vue2Diff(prevChildren, nextChildren, parent) {
2    // ...
3    while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
4      if (oldStartNode.key === newStartNode.key) {
5        // ...
6      } else if (oldEndNode.key === newEndNode.key) {
7        // ...
8      } else if (oldStartNode.key === newEndNode.key) {
9        patch(oldStartNode, newEndNode, parent)
10       parent.insertBefore(oldStartNode.el, oldEndNode.el.nextSibling)
11
12       oldStartIndex++
13       newEndIndex--
14       oldStartNode = prevChildren[oldStartIndex]
15       newEndNode = nextChildren[newEndIndex]
16     } else if (oldEndNode.key === newStartNode.key) {
17       //...

```

```

18     }
19     }
20 }

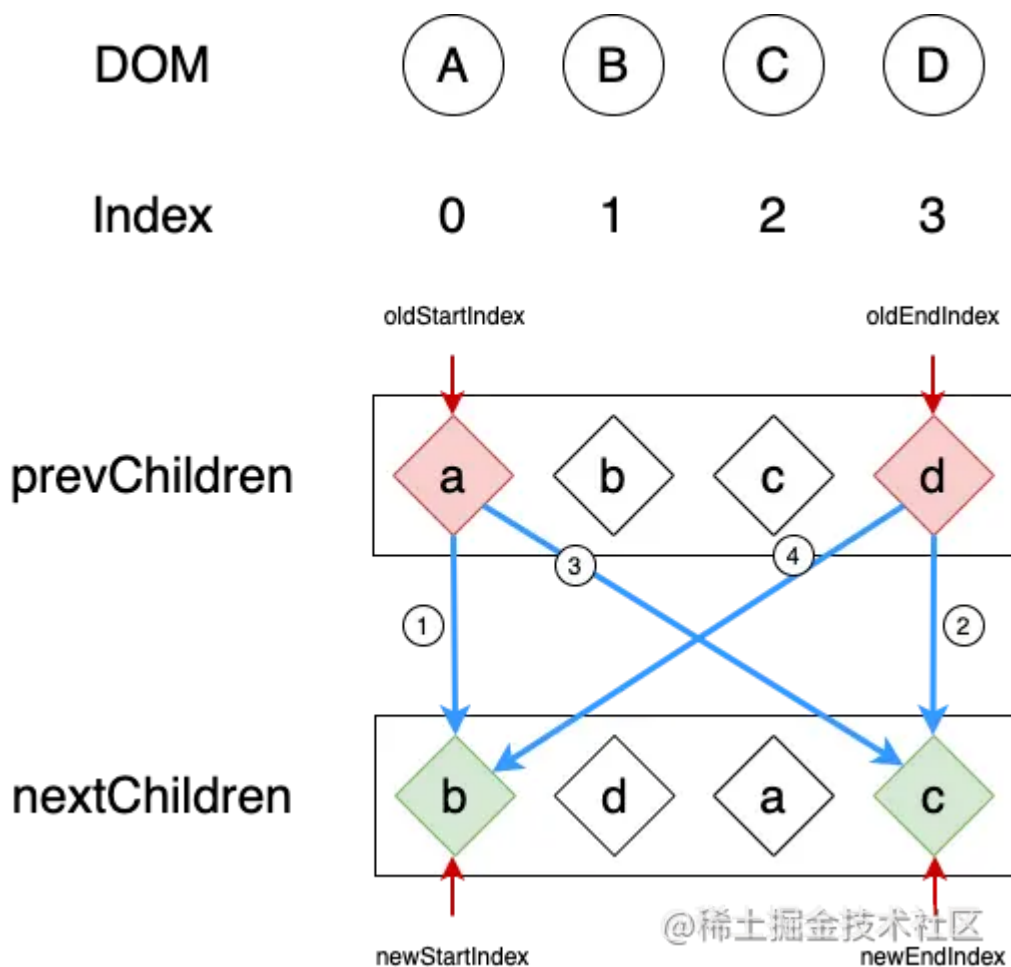
```



进入最后一个循环。在第一步旧列表头节点 `oldStartNode` 与新列表头节点 `newStartNode` 位置相同，所以啥也不用做。然后结束循环。

非理想情况

上文中有一个特殊情况，当四次对比都没找到复用节点时，我们只能拿新列表的第一个节点去旧列表中找到与其key相同的节点。

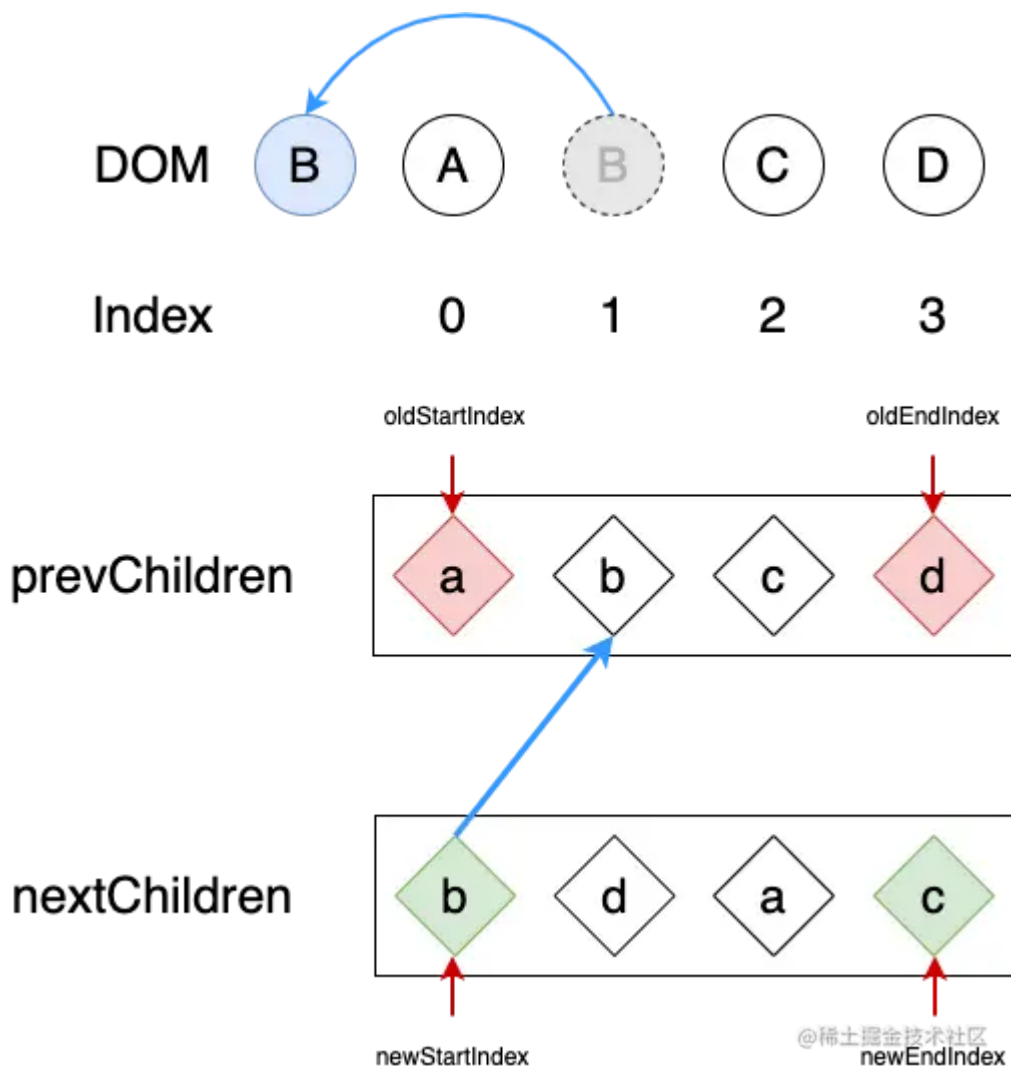


```

1  function vue2Diff(prevChildren, nextChildren, parent) {
2    //...
3    while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
4      if (oldStartNode.key === newStartNode.key) {
5        //...
6      } else if (oldEndNode.key === newEndNode.key) {
7        //...
8      } else if (oldStartNode.key === newEndNode.key) {
9        //...
10     } else if (oldEndNode.key === newStartNode.key) {
11       //...
12     } else {
13       // 在旧列表中找到 和新列表头节点key 相同的节点
14       let newKey = newStartNode.key,
15         oldIndex = prevChildren.findIndex(child => child.key === newKey);
16
17     }
18   }
19 }

```

找节点的时候其实会有两种情况：一种在旧列表中找到了，另一种情况是没找到。



当我们在旧列表中找到对应的VNode，我们只需要将找到的节点的DOM元素，移动到开头就可以了。这里的逻辑其实和第四步的逻辑是一样的，只不过第四步是移动的尾节点，这里是移动找到的节点。DOM移动后，由我们将旧列表中的节点改为undefined，这是至关重要的一步，因为我们已经做了节点的移动了所以我们不需要进行再次的对比了。最后我们将头指针newStartIndex向后移一位。

```

1  function vue2Diff(prevChildren, nextChildren, parent) {
2    //...
3    while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
4      if (oldStartNode.key === newStartNode.key) {
5        //...
6      } else if (oldEndNode.key === newEndNode.key) {
7        //...
8      } else if (oldStartNode.key === newEndNode.key) {
9        //...
10     } else if (oldEndNode.key === newStartNode.key) {
11       //...
12     } else {
13       // 在旧列表中找到 和新列表头节点key 相同的节点
14       let newKey = newStartNode.key,
15         oldIndex = prevChildren.findIndex(child => child.key === newKey);
16

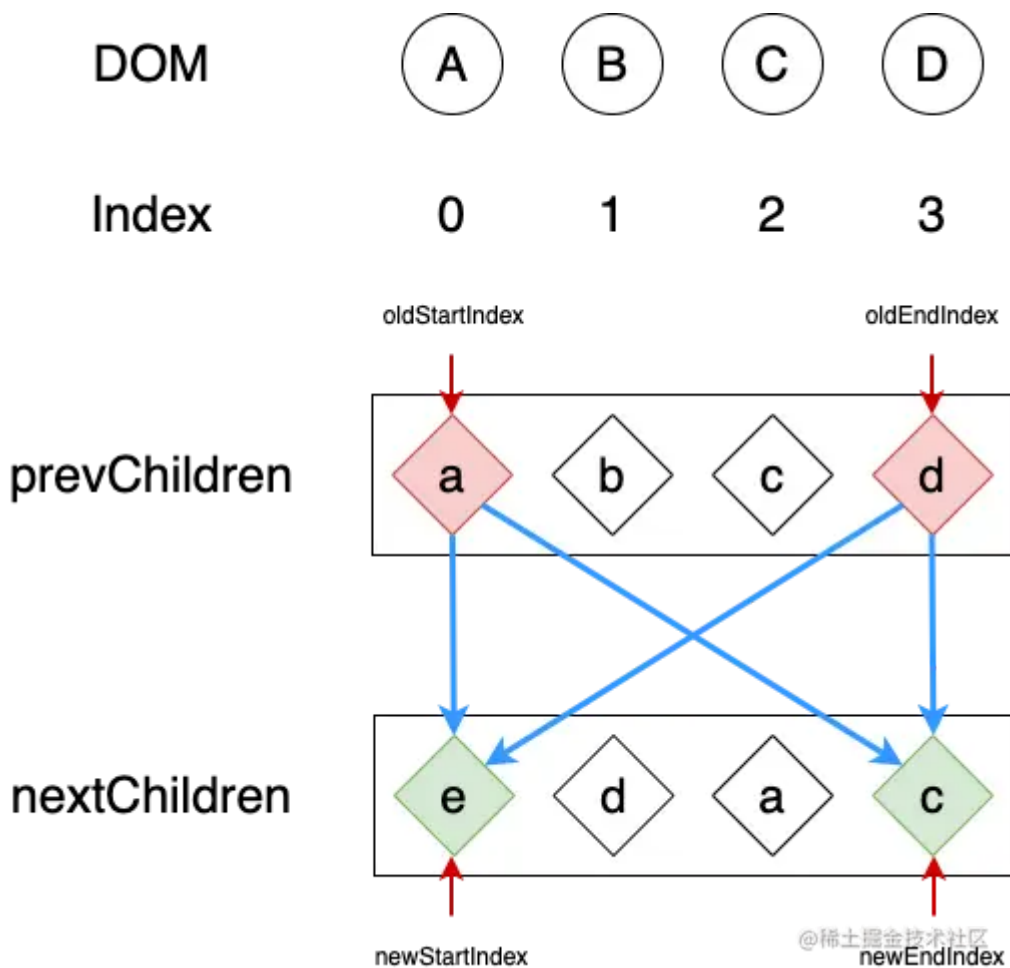
```

```

17     if (oldIndex > -1) {
18         let oldNode = prevChildren[oldIndex];
19         patch(oldNode, newStartNode, parent)
20         parent.insertBefore(oldNode.el, oldStartNode.el)
21         prevChildren[oldIndex] = undefined
22     }
23     newStartNode = nextChildren[++newStartIndex]
24 }
25 }
26 }

```

如果在旧列表中没有找到复用节点，就直接创建一个新的节点放到最前面就可以了，然后后移头指针 `newStartIndex`。



```

1  function vue2Diff(prevChildren, nextChildren, parent) {
2      //...
3      while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
4          if (oldStartNode.key === newStartNode.key) {
5              //...
6          } else if (oldEndNode.key === newEndNode.key) {
7              //...
8          } else if (oldStartNode.key === newEndNode.key) {

```

```

9      //...
10     } else if (oldEndNode.key === newStartNode.key) {
11       //...
12     } else {
13       // 在旧列表中找到 和新列表头节点key 相同的节点
14       let newKey = newStartNode.key,
15         oldIndex = prevChildren.findIndex(child => child.key === newKey);
16
17       if (oldIndex > -1) {
18         let oldNode = prevChildren[oldIndex];
19         patch(oldNode, newStartNode, parent)
20         parent.insertBefore(oldNode.el, oldStartNode.el)
21         prevChildren[oldIndex] = undefined
22       } else {
23         mount(newStartNode, parent, oldStartNode.el)
24       }
25       newStartNode = nextChildren[++newStartIndex]
26     }
27   }
28 }

```

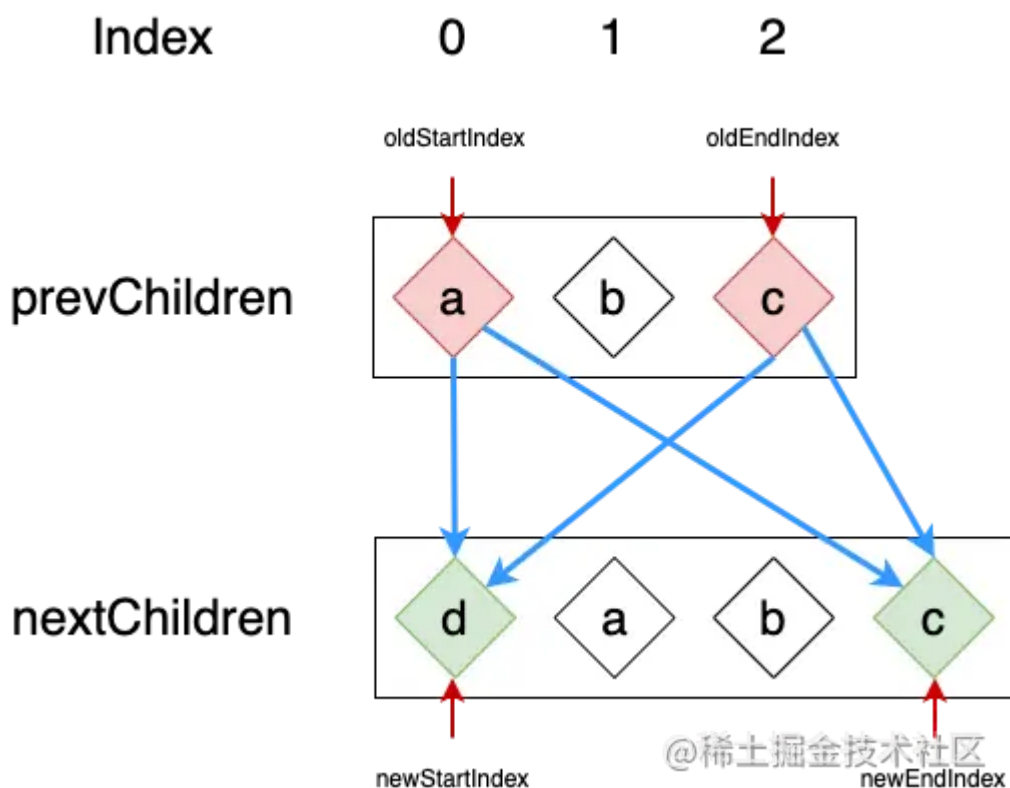
最后当旧列表遍历到undefined时就跳过当前节点。

```

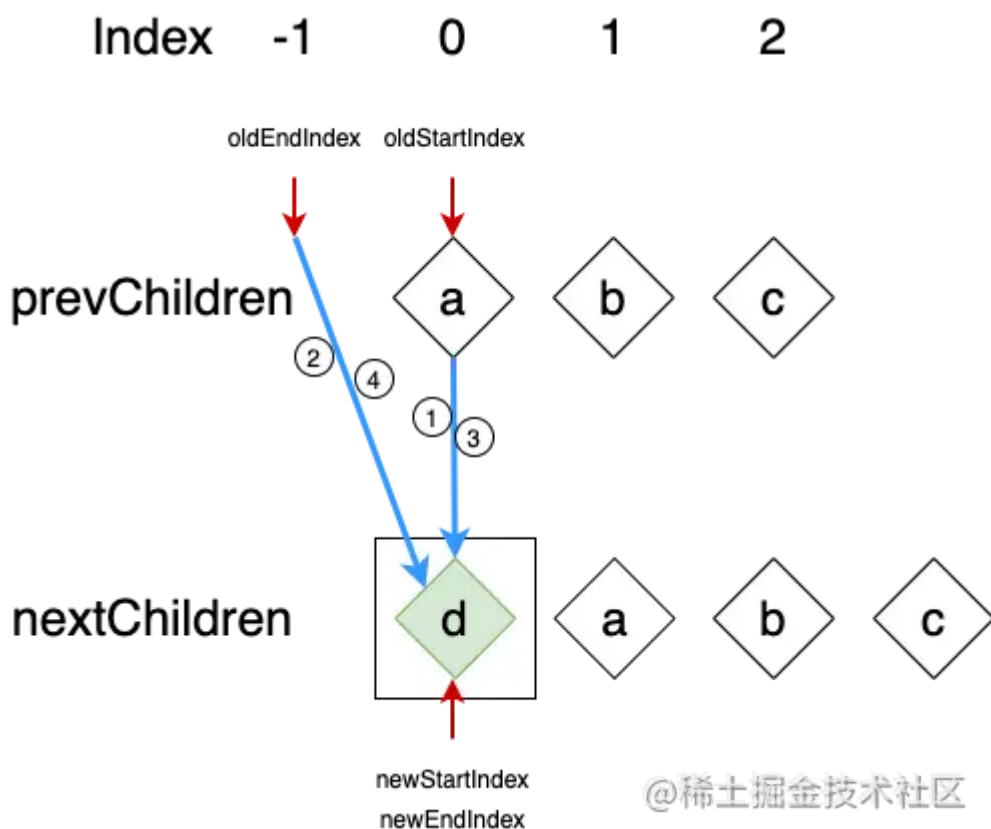
1  function vue2Diff(prevChildren, nextChildren, parent) {
2    //...
3    while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
4      if (oldStartNode === undefined) {
5        oldStartNode = prevChildren[++oldStartIndex]
6      } else if (oldEndNode === undefined) {
7        oldEndNode = prevChildren[--oldEndIndex]
8      } else if (oldStartNode.key === newStartNode.key) {
9        //...
10       } else if (oldEndNode.key === newEndNode.key) {
11         //...
12       } else if (oldStartNode.key === newEndNode.key) {
13         //...
14       } else if (oldEndNode.key === newStartNode.key) {
15         //...
16       } else {
17         // ...
18       }
19     }
20   }

```

添加节点



针对上述例子，几次循环都是尾节点相同，尾指针一直向前移动，直到循环结束；



此时 `oldEndIndex` 以及小于了 `oldStartIndex`，但是新列表中还有剩余的节点，我们只需要将剩余的节点依次插入到 `oldStartNode` 的DOM之前就可以了。为什么是插入 `oldStartNode` 之前

呢？原因是剩余的节点在新列表的位置是位于 `oldStartNode` 之前的，如果剩余节点是在 `oldStartNode` 之后，`oldStartNode` 就会先行对比

```
1  function vue2Diff(prevChildren, nextChildren, parent) {
2    //...
3    while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
4      // ...
5    }
6    if (oldEndIndex < oldStartIndex) {
7      for (let i = newStartIndex; i <= newEndIndex; i++) {
8        mount(nextChildren[i], parent, prevStartNode.el)
9      }
10   }
11 }
```

移除节点

当新列表的 `newEndIndex` 小于 `newStartIndex` 时，我们将旧列表剩余的节点删除即可。这里我们需要注意，旧列表的`undefind`。在第二小节中我们提到过，当头尾节点都不相同时，我们会去旧列表中新列表的第一个节点，移动完DOM节点后，将旧列表的那个节点改为`undefind`。所以我们在最后的删除时，需要注意这些`undefind`，遇到的话跳过当前循环即可。

```
1  function vue2Diff(prevChildren, nextChildren, parent) {
2    //...
3    while (oldStartIndex <= oldEndIndex && newStartIndex <= newEndIndex) {
4      // ...
5    }
6    if (oldEndIndex < oldStartIndex) {
7      for (let i = newStartIndex; i <= newEndIndex; i++) {
8        mount(nextChildren[i], parent, prevStartNode.el)
9      }
10   } else if (newEndIndex < newStartIndex) {
11     for (let i = oldStartIndex; i <= oldEndIndex; i++) {
12       if (prevChildren[i]) {
13         partent.removeChild(prevChildren[i].el)
14       }
15     }
16   }
17 }
```

总结

```
1  function vue2diff(prevChildren, nextChildren, parent) {
2    let oldStartIndex = 0,
3        newStartIndex = 0,
4        oldStartIndex = prevChildren.length - 1,
5        newStartIndex = nextChildren.length - 1,
6        oldStartNode = prevChildren[oldStartIndex],
7        oldEndNode = prevChildren[oldStartIndex],
8        newStartNode = nextChildren[newStartIndex],
9        newEndNode = nextChildren[newStartIndex];
10   while (oldStartIndex <= oldStartIndex && newStartIndex <= newStartIndex) {
11     if (oldStartNode === undefined) {
12       oldStartNode = prevChildren[++oldStartIndex]
13     } else if (oldEndNode === undefined) {
14       oldEndNode = prevChildren[--oldStartIndex]
15     } else if (oldStartNode.key === newStartNode.key) {
16       patch(oldStartNode, newStartNode, parent)
17
18       oldStartIndex++
19       newStartIndex++
20       oldStartNode = prevChildren[oldStartIndex]
21       newStartNode = nextChildren[newStartIndex]
22     } else if (oldEndNode.key === newEndNode.key) {
23       patch(oldEndNode, newEndNode, parent)
24
25       oldStartIndex--
26       newStartIndex--
27       oldEndNode = prevChildren[oldStartIndex]
28       newEndNode = nextChildren[newStartIndex]
29     } else if (oldStartNode.key === newEndNode.key) {
30       patch(oldStartNode, newEndNode, parent)
31       parent.insertBefore(oldStartNode.el, oldEndNode.el.nextSibling)
32       oldStartIndex++
33       newStartIndex--
34       oldStartNode = prevChildren[oldStartIndex]
35       newEndNode = nextChildren[newStartIndex]
36     } else if (oldEndNode.key === newStartNode.key) {
37       patch(oldEndNode, newStartNode, parent)
38       parent.insertBefore(oldEndNode.el, oldStartNode.el)
39       oldStartIndex--
40       newStartIndex++
41       oldEndNode = prevChildren[oldStartIndex]
42       newStartNode = nextChildren[newStartIndex]
```

```

43     } else {
44         let newKey = newStartNode.key,
45             oldIndex = prevChildren.findIndex(child => child && (child.key ===
newKey));
46         if (oldIndex === -1) {
47             mount(newStartNode, parent, oldStartNode.el)
48         } else {
49             let prevNode = prevChildren[oldIndex]
50             patch(prevNode, newStartNode, parent)
51             parent.insertBefore(prevNode.el, oldStartNode.el)
52             prevChildren[oldIndex] = undefined
53         }
54         newStartIndex++
55         newStartNode = nextChildren[newStartIndex]
56     }
57 }
58 if (newStartIndex > newEndIndex) {
59     while (oldStartIndex <= oldEndIndex) {
60         if (!prevChildren[oldStartIndex]) {
61             oldStartIndex++
62             continue
63         }
64         parent.removeChild(prevChildren[oldStartIndex++].el)
65     }
66 } else if (oldStartIndex > oldEndIndex) {
67     while (newStartIndex <= newEndIndex) {
68         mount(nextChildren[newStartIndex++], parent, oldStartNode.el)
69     }
70 }
71 }

```

缺点

Vue2 是全量 Diff（当数据发生变化，它就会新生成一个DOM树，并和之前的DOM树进行比较，找到不同的节点然后更新），如果层级很深，很消耗内存；